

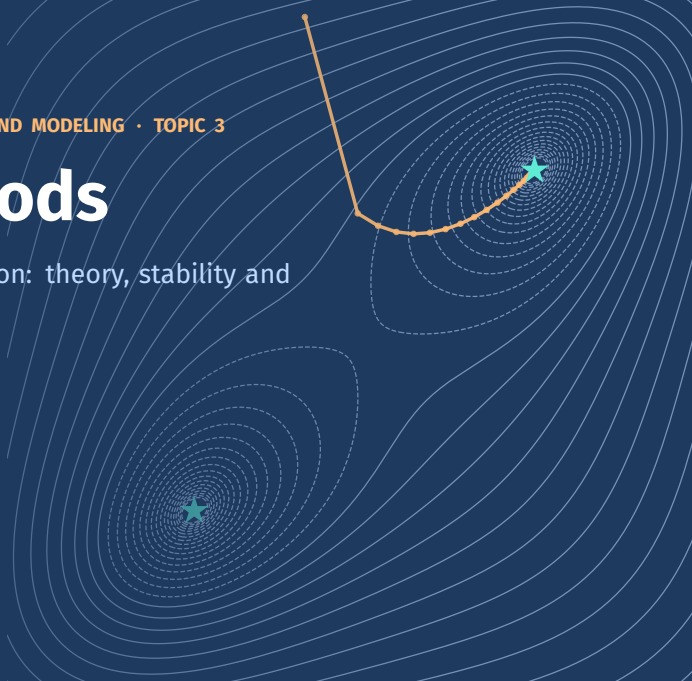
CSE 401 • NUMERICAL ANALYSIS, SIMULATION AND MODELING • TOPIC 3

Gradient Methods

From steepest descent to acceleration: theory, stability and practice

Nawriz Ahmed Turjo

Roll 2105032 • October 2026



Topic 3: Gradient Methods

One idea, $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$, and everything it takes to make it work.

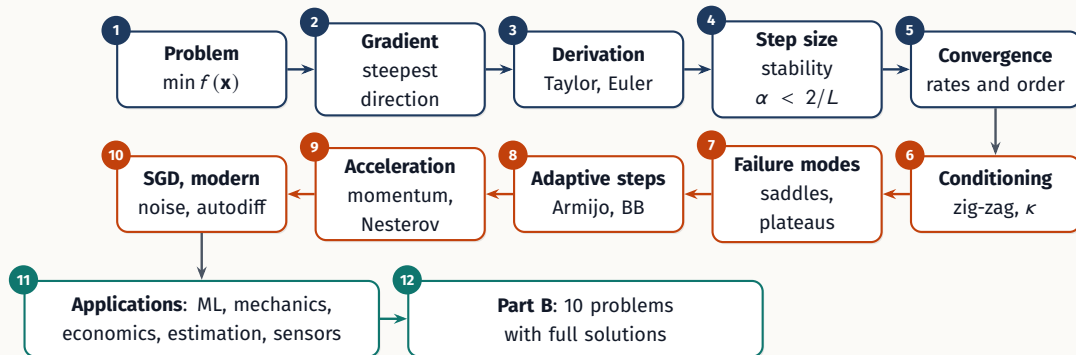
Part A: the ideas

Where the method comes from and why it converges, how big a step can be, why it slows down, how it fails, and how momentum, adaptive steps and SGD fix it.

Part B: the problems

Five mathematical and five real-life problems, each followed straight away by a complete step-by-step solution.

Where we are going



1–5 how it works

6–10 why it struggles, and the fixes

11–12 putting it to use

PART A

The ideas

Where gradient descent comes from, why it works, and why it sometimes does not.

01

The optimization problem

start ● ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ Part B

The problem

Train a model, settle a structure, plan production, fit data. Each asks: **which input makes one number smallest?**

Field	Unknown $\mathbf{x}$	Objective $f(\mathbf{x})$ to minimise
Machine learning	model weights	training loss
Structural mechanics	nodal displacements	potential energy
Economics	production levels	–profit
Parameter estimation	physical constants	sum of squared residuals
Signal processing	filter or estimate	mean-squared error

The common form

$$\min_{\mathbf{x} \in \mathbb{R}^n} f(\mathbf{x}), \quad f : \mathbb{R}^n \rightarrow \mathbb{R} \text{ differentiable} \quad (\text{maximise } P = \text{minimise } -P).$$

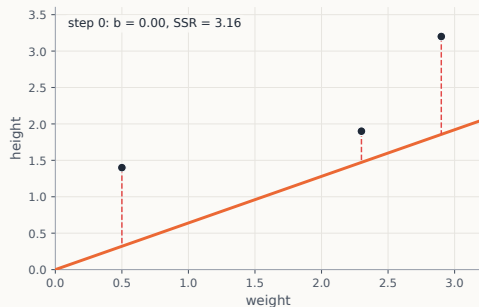
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

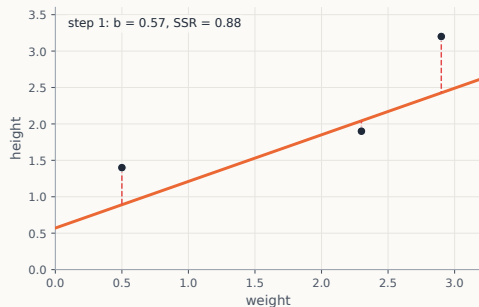
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

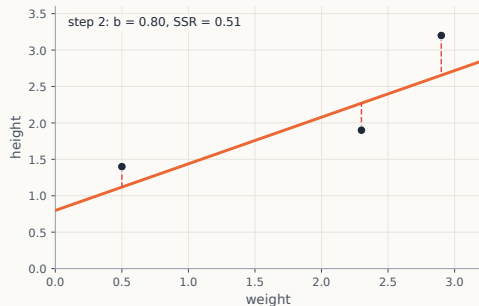
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

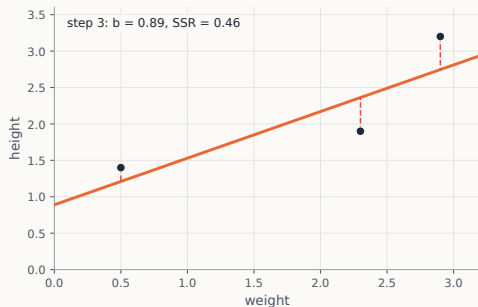
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

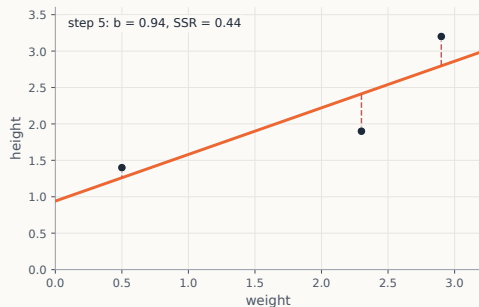
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

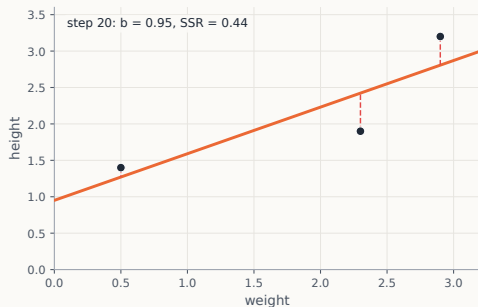
Predict height from weight with a straight line,
 $\hat{y} = b + mx$ (intercept b , slope m). How good is
a line? Add up the squared **residuals**
 $r_i = y_i - (b + mx_i)$:

$$\text{SSR}(b, m) = \sum_{i=1}^n (y_i - b - mx_i)^2.$$

This is the **loss function**. The best line is the one
with the smallest SSR.

Other uses

Linear and logistic regression, t-SNE: each one is
“pick the parameters that make a loss smallest”.



Data: weights 0.5, 2.3, 2.9, heights 1.4, 1.9, 3.2. Each
click shows smaller residuals.

$\mathbf{x}^*$ is a **global minimiser** if $f(\mathbf{x}^*) \leq f(\mathbf{x})$ for every $\mathbf{x}$, and a **local minimiser** if that only holds near $\mathbf{x}^*$.

First-order necessary condition

If $\mathbf{x}^*$ is a local minimiser and f is differentiable, then $\nabla f(\mathbf{x}^*) = 0$.

Proof. Suppose $\mathbf{g} = \nabla f(\mathbf{x}^*) \neq 0$. Then $f(\mathbf{x}^* - t\mathbf{g}) = f(\mathbf{x}^*) - t\|\mathbf{g}\|^2 + o(t) < f(\mathbf{x}^*)$ for small $t > 0$, so $\mathbf{x}^*$ was not a minimiser. ■

- Second order: $\nabla^2 f(\mathbf{x}^*) \succeq 0$ is necessary, and $\nabla f = 0$ with $\nabla^2 f \succ 0$ is sufficient.
- A zero gradient does not guarantee a minimum. $x^4 + y^4 - 4xy$ has a **saddle** at the origin (Problem M1).

- $\nabla f(\mathbf{x}) = 0$ is a system of n *nonlinear* equations, and closed-form solutions are rare.
- Second-order methods use the Hessian, which costs n^2 memory and $O(n^3)$ work per linear solve. They are a separate topic and appear here only for contrast.
- **First-order methods** need only ∇f : $O(n)$ memory and one gradient per iteration. That is cheap enough for $n \sim 10^9$, the size of modern neural networks.

A little history

Cauchy proposed steepest descent in 1847 to solve systems of simultaneous equations, by driving a nonnegative function down to zero [6]. More than 175 years later, variants of the same idea train today's largest models [3].

In this module we

- derive gradient descent and prove how fast it converges,
- find out *when* and *why* it slows down,
- fix it with adaptive steps, momentum and SGD.

02

The gradient: which way is downhill?

start ● ○ ○ ○ ○ ○ ○ ○ ○ ○ ○ Part B

The gradient

- The gradient collects all partial derivatives: $\nabla f(\mathbf{x}) = \left(\frac{\partial f}{\partial x_1}, \dots, \frac{\partial f}{\partial x_n} \right)^\top$.
- The directional derivative along $\mathbf{d}$ tells you how fast f changes if you move that way (chain rule on $t \mapsto f(\mathbf{x} + t\mathbf{d})$):

$$D_{\mathbf{d}}f(\mathbf{x}) = \lim_{t \rightarrow 0} \frac{f(\mathbf{x} + t\mathbf{d}) - f(\mathbf{x})}{t} = \nabla f(\mathbf{x})^\top \mathbf{d}.$$

- First-order Taylor expansion: $f(\mathbf{x} + t\mathbf{d}) = f(\mathbf{x}) + t \nabla f(\mathbf{x})^\top \mathbf{d} + o(t)$.
- So if $\nabla f(\mathbf{x})^\top \mathbf{d} < 0$, a small step along $\mathbf{d}$ lowers f . We call $\mathbf{d}$ a **descent direction**.

The question that starts everything

Of all the unit directions you could step in, which one lowers f the *fastest*?

Theorem: $-\nabla f$ is the steepest way down

The gradient: which way is downhill?

Theorem (steepest descent)

If $\nabla f(\mathbf{x}) \neq 0$, then

$$\min_{\|\mathbf{d}\|=1} \nabla f(\mathbf{x})^T \mathbf{d} = -\|\nabla f(\mathbf{x})\|,$$

and the only minimiser is $\mathbf{d}^\star = -\nabla f(\mathbf{x})/\|\nabla f(\mathbf{x})\|$.

Proof.

Theorem: $-\nabla f$ is the steepest way down

The gradient: which way is downhill?

Theorem (steepest descent)

If $\nabla f(\mathbf{x}) \neq 0$, then

$$\min_{\|\mathbf{d}\|=1} \nabla f(\mathbf{x})^T \mathbf{d} = -\|\nabla f(\mathbf{x})\|,$$

and the only minimiser is $\mathbf{d}^\star = -\nabla f(\mathbf{x})/\|\nabla f(\mathbf{x})\|$.

Proof. By Cauchy-Schwarz,

$$|\nabla f^T \mathbf{d}| \leq \|\nabla f\| \|\mathbf{d}\| = \|\nabla f\|, \text{ so } \nabla f^T \mathbf{d} \geq -\|\nabla f\|.$$

Theorem: $-\nabla f$ is the steepest way down

The gradient: which way is downhill?

Theorem (steepest descent)

If $\nabla f(\mathbf{x}) \neq 0$, then

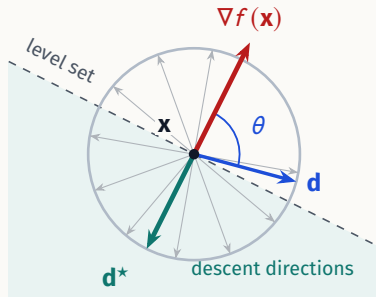
$$\min_{\|\mathbf{d}\|=1} \nabla f(\mathbf{x})^T \mathbf{d} = -\|\nabla f(\mathbf{x})\|,$$

and the only minimiser is $\mathbf{d}^* = -\nabla f(\mathbf{x}) / \|\nabla f(\mathbf{x})\|$.

Proof. By Cauchy-Schwarz,

$$|\nabla f^T \mathbf{d}| \leq \|\nabla f\| \|\mathbf{d}\| = \|\nabla f\|, \text{ so } \nabla f^T \mathbf{d} \geq -\|\nabla f\|.$$

Cauchy-Schwarz is an equality only when $\mathbf{d}$ is parallel to ∇f . Taking the minus sign, $\mathbf{d} = -\nabla f / \|\nabla f\|$ reaches the bound. ■



$D_{\mathbf{d}}f = \|\nabla f\| \cos \theta$, which is smallest at $\theta = \pi$.
Every arrow in the green half-plane goes downhill; $\mathbf{d}^*$ goes down fastest.

The gradient is perpendicular to the contours

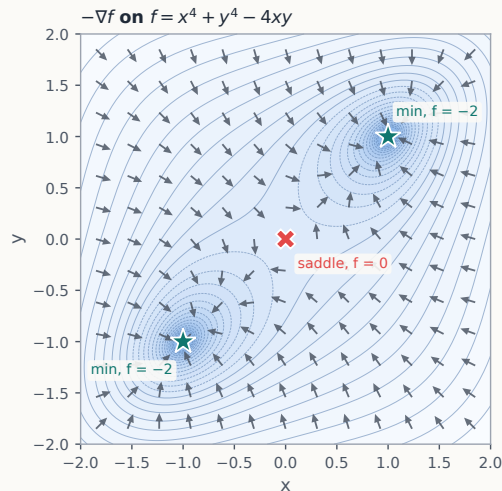
The gradient: which way is downhill?

Take any smooth curve $\gamma(t)$ that stays on the level set $\{f = c\}$, with $\gamma(0) = \mathbf{x}$. Then

$$f(\gamma(t)) = c \Rightarrow \nabla f(\mathbf{x})^\top \gamma'(0) = 0.$$

So $\nabla f(\mathbf{x})$ is perpendicular to every direction along the contour.

- The arrows show $-\nabla f$ (normalised) for $f = x^4 + y^4 - 4xy$.
- Look at how they cross every contour at a right angle, run away from the saddle along $(1, 1)$, and pour into the two minima.



Swap the Euclidean norm for $\|\mathbf{d}\|_{\mathbf{P}} = \sqrt{\mathbf{d}^T \mathbf{P} \mathbf{d}}$ with $\mathbf{P} \succ 0$, and the answer changes:

$$\min_{\mathbf{d}^T \mathbf{P} \mathbf{d} \leq 1} \nabla f^T \mathbf{d} \implies \mathbf{d}^* \propto -\mathbf{P}^{-1} \nabla f(\mathbf{x}).$$

Proof. Substitute $\mathbf{d} = \mathbf{P}^{-1/2} \mathbf{z}$ with $\|\mathbf{z}\| \leq 1$. The objective becomes $(\mathbf{P}^{-1/2} \nabla f)^T \mathbf{z}$, so the previous theorem gives $\mathbf{z}^* \propto -\mathbf{P}^{-1/2} \nabla f$ and therefore $\mathbf{d}^* \propto -\mathbf{P}^{-1} \nabla f$. ■

Choice of $\mathbf{P}$	What you get
$\mathbf{I}$	plain gradient descent
a diagonal scaling	feature standardisation, Adam-type methods (Section 13)
$\nabla^2 f(\mathbf{x})$	the Newton direction (a different topic, mentioned only)

So there is no single “steepest” direction. It depends on the geometry you pick [4], and choosing that geometry well is what preconditioning does (Section 8).

03

Three ways to arrive at gradient descent



The Taylor model at $\mathbf{x}_k$ is good close to $\mathbf{x}_k$ and gets worse as you move away. So minimise the model plus a penalty for going too far:

$$\mathbf{x}_{k+1} = \arg \min_{\mathbf{x}} \left\{ \underbrace{f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top (\mathbf{x} - \mathbf{x}_k)}_{\text{linear model}} + \underbrace{\frac{1}{2\alpha} \|\mathbf{x} - \mathbf{x}_k\|^2}_{\text{stay close}} \right\}.$$

Set the gradient of the braces to zero:

$$\nabla f(\mathbf{x}_k) + \frac{1}{\alpha} (\mathbf{x} - \mathbf{x}_k) = 0 \implies \boxed{\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)}$$

- α is the **step size** (the “learning rate” in ML). A large α means we trust the linear model further out.
- Change the penalty and you get a different method. With $\frac{1}{2} \|\cdot\|_{\mathbf{P}}^2$ you get preconditioned GD (Section 2).

View 2: minimise an upper bound

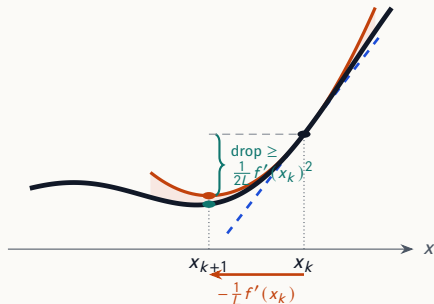
Three ways to arrive at gradient descent

If ∇f is L -Lipschitz, the *descent lemma* (Section 7, Problem M2) says that for every $\mathbf{x}$

$$f(\mathbf{x}) \leq f(\mathbf{x}_k) + \nabla f(\mathbf{x}_k)^\top (\mathbf{x} - \mathbf{x}_k) + \frac{L}{2} \|\mathbf{x} - \mathbf{x}_k\|^2.$$

- That is View 1 with $\alpha = 1/L$, except now the model is an **upper bound** touching f at $\mathbf{x}_k$.
- Its minimiser is $\mathbf{x}_{k+1} = \mathbf{x}_k - \frac{1}{L} \nabla f(\mathbf{x}_k)$.
- $f(\mathbf{x}_{k+1}) \leq$ bound at $\mathbf{x}_{k+1} \leq$ bound at $\mathbf{x}_k = f(\mathbf{x}_k)$, so f **has to go down**.

This trick is known as majorise and minimise (MM).



- f — tangent — upper model (curvature L)
- minimum of the model, • the true $f(x_{k+1})$, lower still.

View 3, the central idea: GD is forward Euler on the gradient flow

The **gradient flow** ODE $\dot{\mathbf{x}}(t) = -\nabla f(\mathbf{x}(t))$ always runs downhill:

$$\frac{d}{dt}f(\mathbf{x}(t)) = \nabla f^T \dot{\mathbf{x}} = -\|\nabla f(\mathbf{x}(t))\|^2 \leq 0.$$

Now discretise it with **forward Euler**, step h : $\mathbf{x}_{k+1} = \mathbf{x}_k + h(-\nabla f(\mathbf{x}_k))$. That is **gradient descent with $\alpha = h$** . Everything we know about Euler's method now applies.

What we know from ODEs	What it means for optimisation
Euler stability interval $ 1 - h\lambda < 1$	the step limit $\alpha < 2/L$ (Section 5)
stiffness (widely spread eigenvalues)	ill-conditioning and the zig-zag (Section 8)
implicit (backward) Euler	the proximal-point method, stable for any step
second-order ODE with friction	heavy-ball momentum (Section 11)
$\ddot{\mathbf{x}} + \frac{3}{t}\dot{\mathbf{x}} + \nabla f = 0$	Nesterov acceleration [20]

THE CENTRAL IDEA

**Gradient descent is forward Euler
on the gradient flow.**

So the step limit is a stability limit, and ill-conditioning is stiffness.

Algorithm: gradient descent

Input: $\mathbf{x}_0$, step $\alpha > 0$, tolerances ε_{rel} , ε_{abs} , k_{max}

for $k = 0, 1, 2, \dots, k_{\text{max}} - 1$:

$\mathbf{g}_k \leftarrow \nabla f(\mathbf{x}_k)$

if $\|\mathbf{g}_k\| \leq \varepsilon_{\text{rel}} \|\mathbf{g}_0\| + \varepsilon_{\text{abs}}$: **stop**

$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \mathbf{g}_k$

return $\mathbf{x}_k$

When should it stop?

- A small $\|\nabla f\|$ is the natural test. For strongly convex f it even bounds the error:
 $\|\mathbf{x} - \mathbf{x}^*\| \leq \|\nabla f(\mathbf{x})\| / \mu$ (Section 6).
- Stopping only because $|f_{k+1} - f_k|$ is tiny is a trap. On a plateau f hardly changes while $\mathbf{x}$ is still far from the answer (Problem A4).
- Always cap the iteration count and use a relative tolerance, because floating point limits how small the gradient can get (Section 9).

One parameter by hand: the line fit, with numbers

Fix the slope at $m = 0.64$ and let GD find the intercept b , starting from $b_0 = 0$ with learning rate $\alpha = 0.1$.

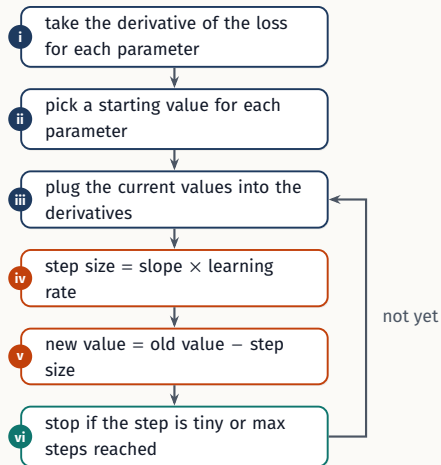
$$\frac{d \text{SSR}}{db} = -2 \sum_i (y_i - b - 0.64 x_i), \quad \text{step} = \frac{d \text{SSR}}{db} \times \alpha, \quad b_{\text{new}} = b_{\text{old}} - \text{step}.$$

step k	b_k	slope $d\text{SSR}/db$	step size	$\text{SSR}(b_{k+1})$
0	0	-5.704	-0.5704	0.878
1	0.5704	-2.282	-0.2282	0.514
2	0.7986	-0.913	-0.0913	0.456
3	0.8898	-0.365	-0.0365	0.446

Each step is about 0.4× the last (start $\text{SSR} = 3.156$), and b settles at $b^* = 0.951$.

Steps shrink because the slope flattens near the minimum.

Several parameters: the same recipe, once per parameter



For $\hat{y} = b + mx$ both parameters move:

$$\frac{\partial \text{SSR}}{\partial b} = -2 \sum_i (y_i - b - mx_i),$$

$$\frac{\partial \text{SSR}}{\partial m} = -2 \sum_i x_i (y_i - b - mx_i).$$

In general, for every w_j at once:

$$w_j \leftarrow w_j - \alpha \frac{\partial f}{\partial w_j}$$

Stop when the step size is near zero (say $< 10^{-3}$) or after a maximum number of steps.

```
import numpy as np

def gradient_descent(grad, x0, alpha,
                    rtol=1e-8, maxit=10_000):
    x = np.asarray(x0, dtype=float).copy()
    g0 = np.linalg.norm(grad(x))
    for k in range(maxit):
        g = grad(x)
        if np.linalg.norm(g) <= rtol * g0:
            break
        x -= alpha * g      #  $O(n)$  update
    return x, k

# least squares:  $f(w) = ||Xw - y||^2 / (2m)$ 
def ls_grad(w, X, y):
    return X.T @ (X @ w - y) / len(y)
```

What one iteration costs

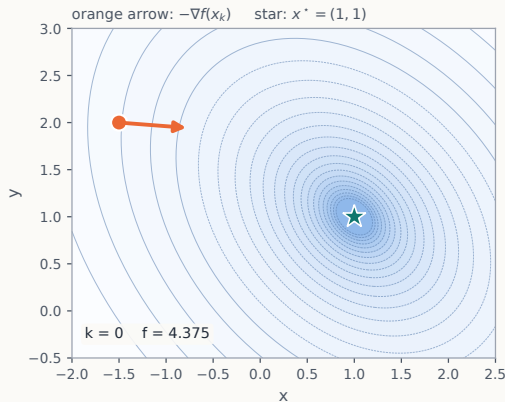
- One gradient and an $O(n)$ vector update, with $O(n)$ memory.
- For least squares with $\mathbf{X} \in \mathbb{R}^{m \times n}$, that is two matrix-vector products, $O(mn)$. Nothing gets factorised.
- Written with array operations (BLAS, GPUs) instead of Python loops, the same code runs for $n = 2$ or for millions of variables.
- Total cost is (number of iterations) $\times$ (cost of ∇f). The rest of the module is about that first factor.

04

Watching it move

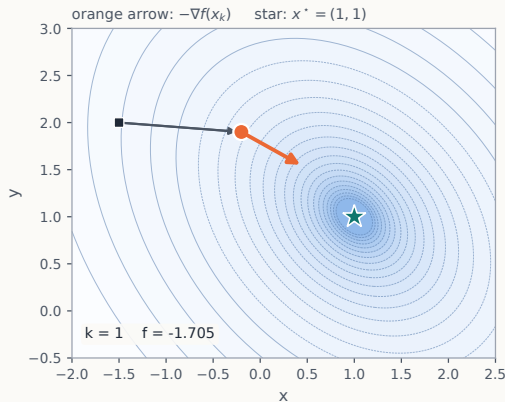
start ● — ● — ● — ○ — ○ — ○ — ○ — ○ — ○ — ○ — ○ — ○ — Part B

Derivation



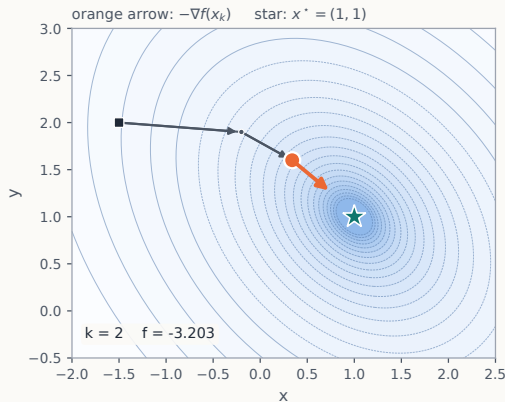
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$,
from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.



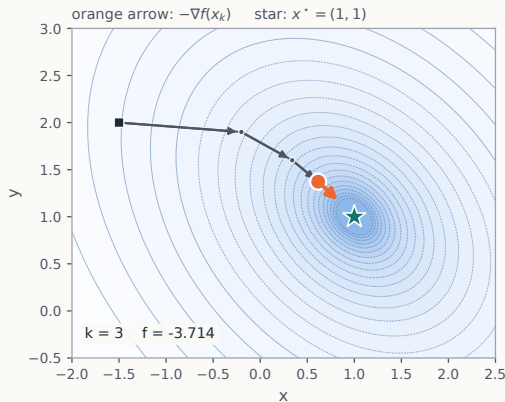
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$,
from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.



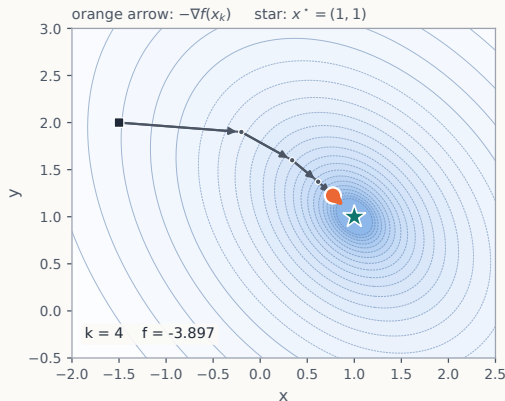
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$,
from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.



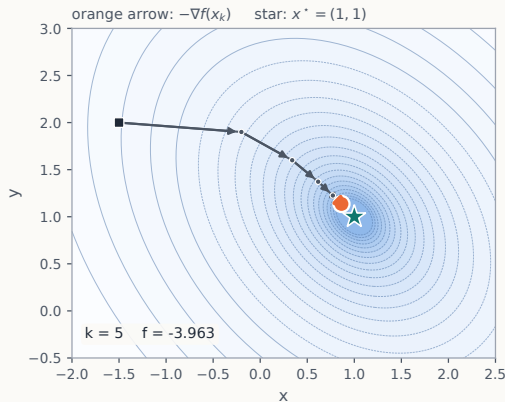
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$,
from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.
- The steps shrink on their own as $\nabla f \rightarrow 0$, with α fixed.



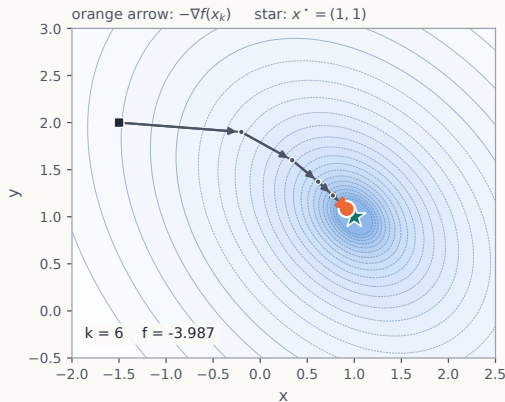
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$, from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.
- The steps shrink on their own as $\nabla f \rightarrow 0$, with α fixed.



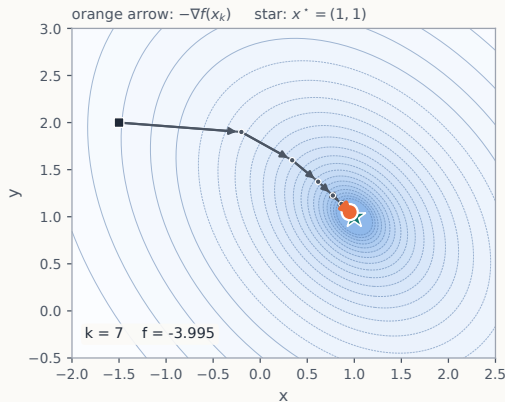
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$,
from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.
- The steps shrink on their own as $\nabla f \rightarrow 0$, with α fixed.
- The path bends: on ellipses, $-\nabla f$ misses $\mathbf{x}^*$.



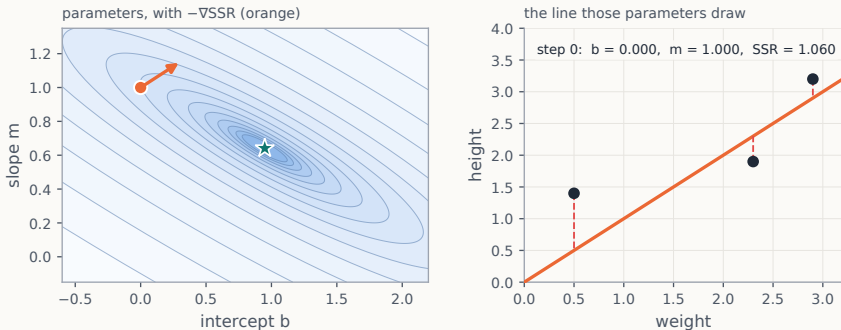
The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$,
from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.
- The steps shrink on their own as $\nabla f \rightarrow 0$, with α fixed.
- The path bends: on ellipses, $-\nabla f$ misses $\mathbf{x}^*$.

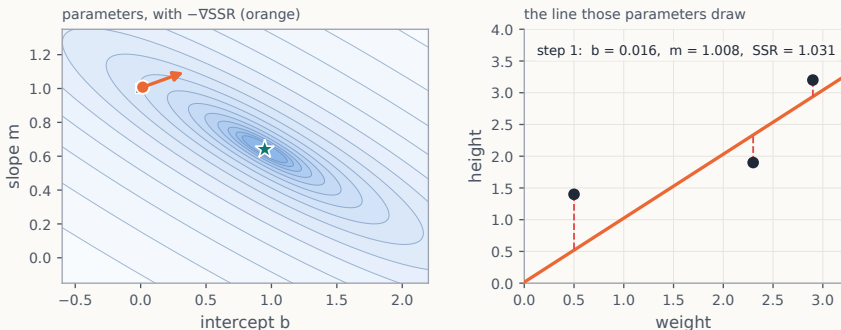


The M3 quadratic, $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = (4, 4)$, from $\mathbf{x}_0 = (-1.5, 2)$, $\alpha = 0.2$.

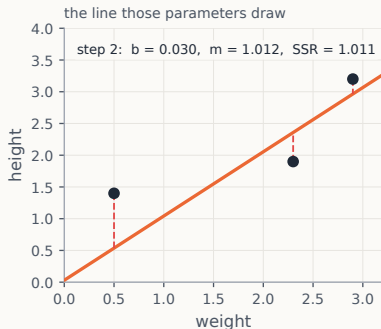
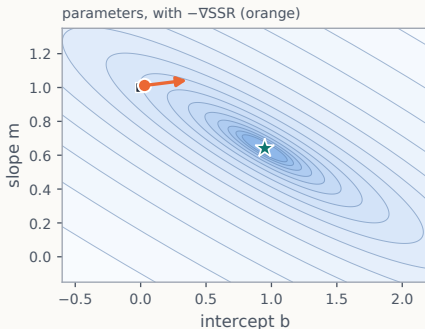
- The orange arrow, $-\nabla f(\mathbf{x}_k)$, leaves at a right angle to the contour.
- The first step is long: $\|\nabla f\|$ is big out here.
- The steps shrink on their own as $\nabla f \rightarrow 0$, with α fixed.
- The path bends: on ellipses, $-\nabla f$ misses $\mathbf{x}^*$.
- After 7 steps $f = -3.995$; the optimum is -4 .



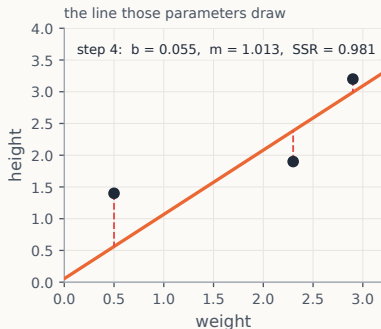
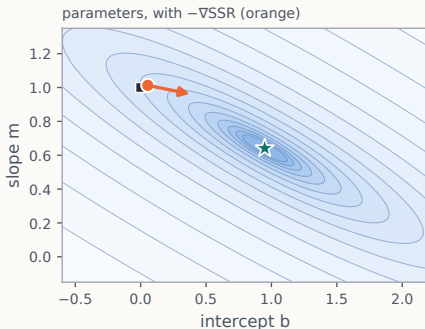
The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



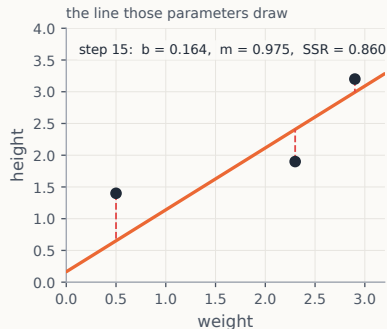
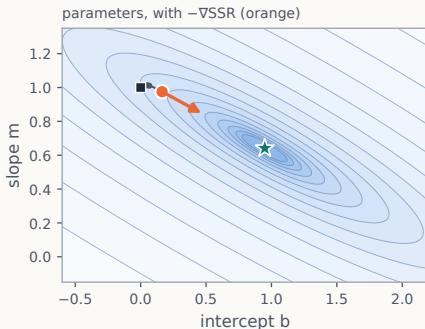
The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



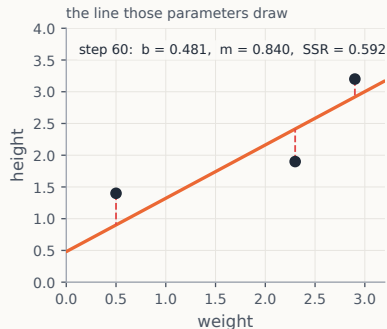
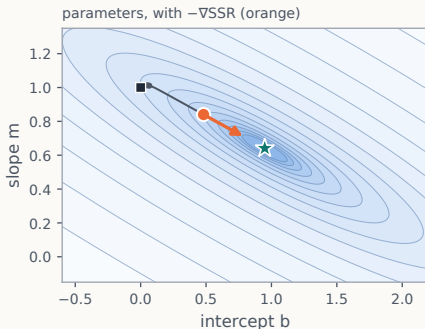
The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



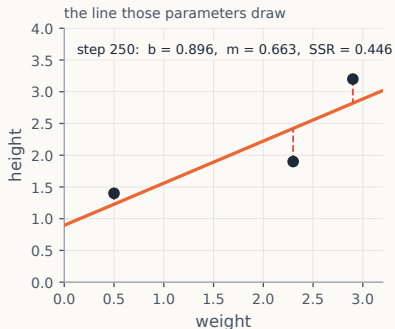
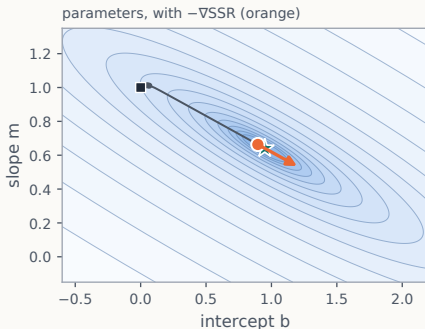
The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



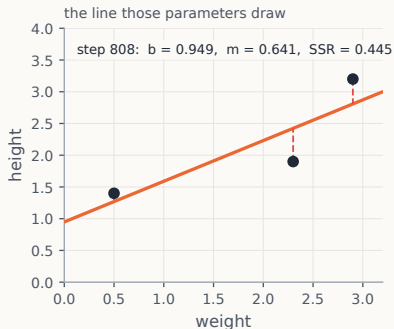
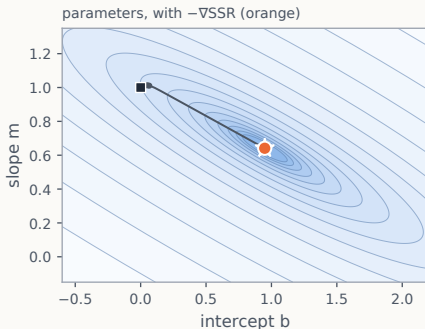
The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



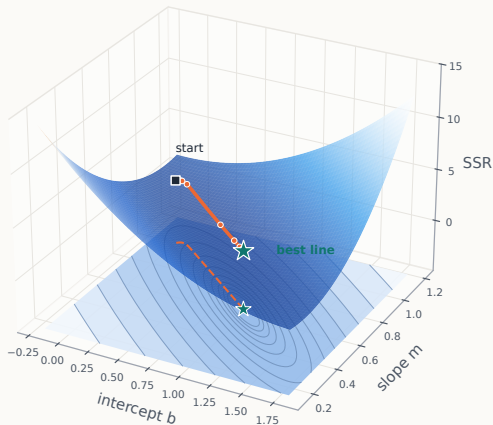
The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.



The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.

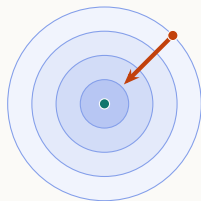


The weight–height data of Section 1, both parameters free, $\alpha = 0.01$ from $(b, m) = (0, 1)$. Left: the point (b, m) walks downhill across the SSR contours. Right: the line that point draws. Click through steps 0, 1, 2, 4, 15, 60, 250, 808.

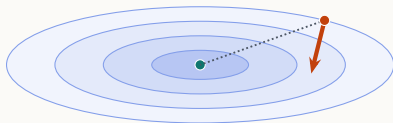


$SSR(b, m)$ is a bowl over the (b, m) plane. Every point of the bowl is one line through the data.

- GD starts at $(0, 1)$ and rolls downhill. The dashed curve is its shadow on the contour map, the path of the previous slide.
- It drops into the valley in a few steps, then creeps along the valley floor, because the valley is long and narrow ($\kappa \approx 29$, see Section 8).
- After 808 steps it stops at $(0.9486, 0.6411)$, the least-squares line $(0.9487, 0.6410)$.



$\kappa = 1$: $-\nabla f$ aims at $\mathbf{x}^*$



$\kappa \gg 1$: $-\nabla f$ misses $\mathbf{x}^*$

- **Round contours** ($\mathbf{A} \propto \mathbf{I}$): $-\nabla f$ points straight at $\mathbf{x}^*$; one well-chosen step lands on it.
- **Stretched contours**: $-\nabla f$ is still perpendicular to the contour, so it *misses* $\mathbf{x}^*$. The iterates bounce across the valley and crawl along it.
- How stretched is measured by the **condition number** $\kappa = \lambda_{\max}/\lambda_{\min}$. It returns in the step limit (Section 5), every rate (Section 7) and the zig-zag (Section 8).

05

How big a step can we take?



Take $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} - \mathbf{b}^T\mathbf{x}$ with $\mathbf{A} \succ 0$ symmetric, so $\nabla f = \mathbf{A}\mathbf{x} - \mathbf{b}$ and $\mathbf{A}\mathbf{x}^\star = \mathbf{b}$. Then

$$\mathbf{e}_{k+1} = \mathbf{x}_{k+1} - \mathbf{x}^\star = (\mathbf{I} - \alpha\mathbf{A})\mathbf{e}_k \implies \mathbf{e}_k = (\mathbf{I} - \alpha\mathbf{A})^k \mathbf{e}_0.$$

Diagonalise $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$ and write $\mathbf{z}_k = \mathbf{Q}^T\mathbf{e}_k$. The coupled iteration falls apart into n independent scalar ones:

$$z_i^{(k)} = (1 - \alpha\lambda_i)^k z_i^{(0)}, \quad i = 1, \dots, n.$$

Why bother with quadratics?

Close to a strict minimiser, $f(\mathbf{x}) \approx f(\mathbf{x}^\star) + \frac{1}{2}(\mathbf{x} - \mathbf{x}^\star)^T \nabla^2 f(\mathbf{x}^\star)(\mathbf{x} - \mathbf{x}^\star)$. So the quadratic analysis with $\mathbf{A} = \nabla^2 f(\mathbf{x}^\star)$ predicts how GD behaves *near the end* on any smooth f (Problem M1(d) uses exactly this).

Theorem: the stable window and the best fixed step

How big a step can we take?

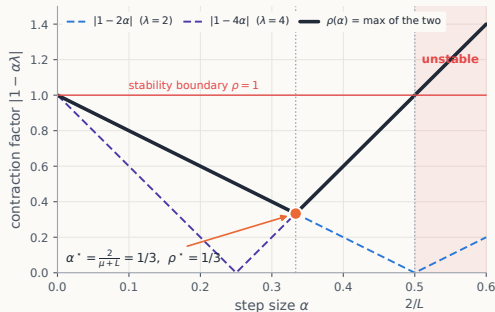
Theorem

GD converges from every $\mathbf{x}_0$ iff $0 < \alpha < 2/\lambda_{\max}$, at rate

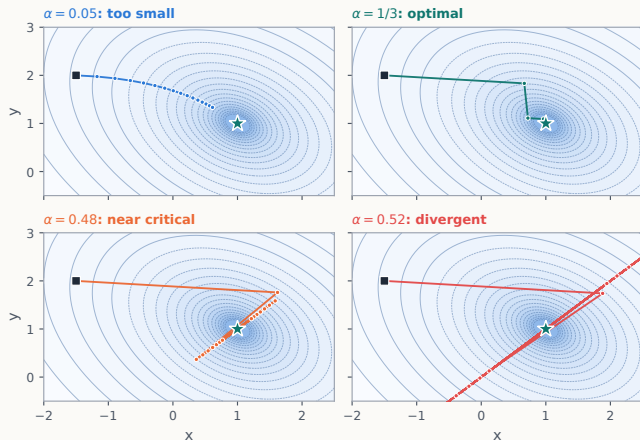
$$\rho(\alpha) = \max\{|1 - \alpha\lambda_{\min}|, |1 - \alpha\lambda_{\max}|\}.$$

The best choice is $\alpha^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$, giving $\rho^* = \frac{\kappa - 1}{\kappa + 1}$ with $\kappa = \frac{\lambda_{\max}}{\lambda_{\min}}$.

Proof. $|1 - \alpha\lambda|$ is convex in λ , so its maximum over $[\lambda_{\min}, \lambda_{\max}]$ sits at an end. Both ends are below 1 iff $\alpha < 2/\lambda_{\max}$. The best α makes the two ends equal: $1 - \alpha\lambda_{\min} = \alpha\lambda_{\max} - 1$. ■



To reach accuracy ε you need about $\frac{\kappa+1}{2} \ln \frac{1}{\varepsilon}$ steps, i.e. $O(\kappa \log \frac{1}{\varepsilon})$.



$\lambda(\mathbf{A}) \in \{2, 4\}$, so GD is stable only for $\alpha < 0.5$.

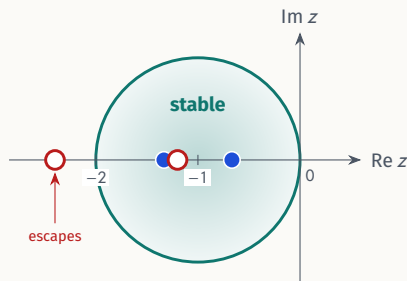
- $\alpha = 0.05$: $\rho = 0.9$. It gets there, eventually.
- $\alpha = \frac{1}{3}$: $\rho = \frac{1}{3}$, the M3 optimum.
- $\alpha = 0.48$: $|1 - 4\alpha| = 0.92$, rattling across the valley.
- $\alpha = 0.52$: $|1 - 4\alpha| = 1.08$, and the swings grow without bound.

Too small wastes time; too large wrecks the run.

For a quadratic the gradient flow is $\dot{\mathbf{e}} = -\mathbf{A}\mathbf{e}$, so each mode is the Dahlquist test equation

$$\dot{y} = \lambda y \text{ with } \lambda = -\lambda_i < 0.$$

- The **exact flow** decays for every $\lambda_i > 0$. Nothing can go wrong in continuous time.
- **Forward Euler** is stable only when $z = h\lambda$ lies in the disk $|1 + z| < 1$. Here $z_i = -\alpha\lambda_i$, so we need $0 < \alpha\lambda_i < 2$, which is exactly $\alpha < 2/\lambda_{\max}$.
- This is **stiffness**: the *fastest* mode ($\lambda_{\max}$) limits the step, while the *slowest* ($\lambda_{\min}$) decides how long you wait. Any explicit scheme pays roughly κ steps.



- $\alpha = \frac{1}{3}$: $z = -\frac{2}{3}, -\frac{4}{3}$, both inside. $\alpha = 0.6$: $z = -1.2, -2.4$, one escapes.

Backward Euler, $\mathbf{x}_{k+1} = \mathbf{x}_k - h\nabla f(\mathbf{x}_{k+1})$, is stable for every h . That is the proximal-point method; the catch is that each step is itself an optimisation problem.

06

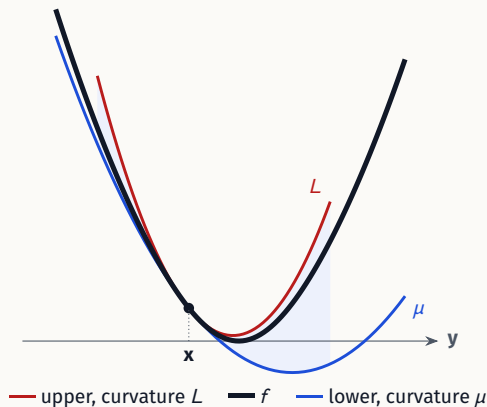
What we need to assume about f



Three assumptions, and which functions satisfy them

Property	Definition	If $f \in C^2$
L -smooth	$\ \nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\ \leq L \ \mathbf{x} - \mathbf{y}\ $	$-L\mathbf{I} \preceq \nabla^2 f \preceq L\mathbf{I}$
convex	$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x})$	$\nabla^2 f \succeq 0$
μ -strongly convex	$f(\mathbf{y}) \geq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{\mu}{2} \ \mathbf{y} - \mathbf{x}\ ^2$	$\nabla^2 f \succeq \mu\mathbf{I}$

Example	L -smooth?	convex?	strongly convex?
$\frac{1}{2}\mathbf{x}^\top \mathbf{A}\mathbf{x} - \mathbf{b}^\top \mathbf{x}, \mathbf{A} \succ 0$	$\checkmark \quad L = \lambda_{\max}$	$\checkmark$	$\checkmark \quad \mu = \lambda_{\min}$
logistic loss $\log(1 + e^{-y\mathbf{a}^\top \mathbf{x}})$	$\checkmark$	$\checkmark$	$\times$
x^4	$\times$ (not globally)	$\checkmark$	$\times$ (flat at 0)
$x^4 + y^4 - 4xy$ (M1)	$\times$ (not globally)	$\times$	$\times$



If f is L -smooth and μ -strongly convex, then at every point it lives inside the shaded corridor between two parabolas.

Minimising the lower parabola over $\mathbf{y}$ gives

$$f^{\star} \geq f(\mathbf{x}) - \frac{1}{2\mu} \|\nabla f(\mathbf{x})\|^2,$$

and three useful facts follow:

- The gradient *certifies* how close you are:
 $f(\mathbf{x}) - f^{\star} \leq \frac{\|\nabla f\|^2}{2\mu}$ and $\|\mathbf{x} - \mathbf{x}^{\star}\| \leq \frac{\|\nabla f(\mathbf{x})\|}{\mu}.$
- There is exactly one minimiser.
- The upper parabola is where the step $1/L$ comes from (Section 3, View 2).

- $\kappa \geq 1$ compares the steepest curvature with the flattest. For a quadratic, the contour ellipses are $\sqrt{\kappa}$ times longer than they are wide.
- Keep an eye on it: κ shows up in *every* rate from here on. GD needs $O(\kappa \log \frac{1}{\epsilon})$ iterations, accelerated methods $O(\sqrt{\kappa} \log \frac{1}{\epsilon})$.

Problem	κ	Where the ill-conditioning comes from
M3 quadratic	2	nowhere, it is a friendly problem
A1 regression, raw feature	46.1	the columns $[x, 1]$ are correlated
A1 after centring	1.5	preprocessing removed it
A2 spring chain, $k_1 = 1000$	1002	one very stiff member next to a soft one
Rosenbrock at $(1, 1)$	≈ 2508	curved valley ($\lambda = 1001.6, 0.399$)

07

How fast does it converge?



Descent lemma (proved in full in Problem M2)

If ∇f is L -Lipschitz: $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$.

Proof idea.

Descent lemma (proved in full in Problem M2)

If ∇f is L -Lipschitz: $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$.

Proof idea. $f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) = \int_0^1 [\nabla f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - \nabla f(\mathbf{x})]^\top (\mathbf{y} - \mathbf{x}) dt,$

Descent lemma (proved in full in Problem M2)

If ∇f is L -Lipschitz: $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$.

Proof idea. $f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) = \int_0^1 [\nabla f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - \nabla f(\mathbf{x})]^\top (\mathbf{y} - \mathbf{x}) dt$, and Cauchy-Schwarz with the Lipschitz bound turns this into $\leq \int_0^1 Lt \|\mathbf{y} - \mathbf{x}\|^2 dt = \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$. ■

Descent lemma (proved in full in Problem M2)

If ∇f is L -Lipschitz: $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$.

Proof idea. $f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) = \int_0^1 [\nabla f(\mathbf{x} + t(\mathbf{y} - \mathbf{x})) - \nabla f(\mathbf{x})]^\top (\mathbf{y} - \mathbf{x}) dt$, and Cauchy-Schwarz with the Lipschitz bound turns this into $\leq \int_0^1 Lt \|\mathbf{y} - \mathbf{x}\|^2 dt = \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2$. ■

Plug in $\mathbf{y} = \mathbf{x}_k - \alpha \mathbf{g}_k$:

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \alpha \left(1 - \frac{\alpha L}{2}\right) \|\mathbf{g}_k\|^2 \stackrel{\alpha=1/L}{=} f(\mathbf{x}_k) - \frac{1}{2L} \|\mathbf{g}_k\|^2.$$

Each step buys a guaranteed drop of $\|\mathbf{g}_k\|^2 / (2L)$. Every rate that follows comes from linking $\|\mathbf{g}_k\|$ to how far we still are from the optimum.

Theorem (L -smooth, bounded below, $\alpha = 1/L$)

$$\min_{0 \leq k < K} \|\nabla f(\mathbf{x}_k)\| \leq \sqrt{\frac{2L(f(\mathbf{x}_0) - f^*)}{K}} = O(1/\sqrt{K}).$$

Proof. Add up the per-step drops for $k = 0, \dots, K - 1$. The total cannot exceed $f(\mathbf{x}_0) - f(\mathbf{x}_K) \leq f(\mathbf{x}_0) - f^*$ (details in Problem M2). ■

- All we get is a point with a *small gradient*. It could be a saddle or a poor local minimum.
- Reaching $\|\nabla f\| \leq \varepsilon$ takes $O(L(f_0 - f^*)/\varepsilon^2)$ iterations, which is slow. That is what you pay for assuming nothing.
- Deep learning lives here. The faster rates on the next slides all need extra structure.

Theorem (L -smooth, convex, $\alpha = 1/L$)

$$f(\mathbf{x}_k) - f^\star \leq \frac{L \|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2k}.$$

Proof.

Theorem (L -smooth, convex, $\alpha = 1/L$)

$$f(\mathbf{x}_k) - f^\star \leq \frac{L \|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2k}.$$

Proof. Convexity gives $f(\mathbf{x}_k) \leq f^\star + \mathbf{g}_k^\top (\mathbf{x}_k - \mathbf{x}^\star)$. Combine that with the guaranteed drop:

$$f(\mathbf{x}_{k+1}) - f^\star \leq \mathbf{g}_k^\top (\mathbf{x}_k - \mathbf{x}^\star) - \frac{1}{2L} \|\mathbf{g}_k\|^2 = \frac{L}{2} \left(\|\mathbf{x}_k - \mathbf{x}^\star\|^2 - \|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 \right).$$

Theorem (L -smooth, convex, $\alpha = 1/L$)

$$f(\mathbf{x}_k) - f^\star \leq \frac{L \|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2k}.$$

Proof. Convexity gives $f(\mathbf{x}_k) \leq f^\star + \mathbf{g}_k^\top (\mathbf{x}_k - \mathbf{x}^\star)$. Combine that with the guaranteed drop:

$$f(\mathbf{x}_{k+1}) - f^\star \leq \mathbf{g}_k^\top (\mathbf{x}_k - \mathbf{x}^\star) - \frac{1}{2L} \|\mathbf{g}_k\|^2 = \frac{L}{2} \left(\|\mathbf{x}_k - \mathbf{x}^\star\|^2 - \|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 \right).$$

(The equality is just $\|\mathbf{x}_k - \mathbf{x}^\star - \mathbf{g}_k/L\|^2$ expanded.) Summing over $k = 0, \dots, K-1$, the right side telescopes:

$$\sum_{k=1}^K (f(\mathbf{x}_k) - f^\star) \leq \frac{L}{2} \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

Theorem (L -smooth, convex, $\alpha = 1/L$)

$$f(\mathbf{x}_k) - f^\star \leq \frac{L \|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{2k}.$$

Proof. Convexity gives $f(\mathbf{x}_k) \leq f^\star + \mathbf{g}_k^\top (\mathbf{x}_k - \mathbf{x}^\star)$. Combine that with the guaranteed drop:

$$f(\mathbf{x}_{k+1}) - f^\star \leq \mathbf{g}_k^\top (\mathbf{x}_k - \mathbf{x}^\star) - \frac{1}{2L} \|\mathbf{g}_k\|^2 = \frac{L}{2} \left(\|\mathbf{x}_k - \mathbf{x}^\star\|^2 - \|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 \right).$$

(The equality is just $\|\mathbf{x}_k - \mathbf{x}^\star - \mathbf{g}_k/L\|^2$ expanded.) Summing over $k = 0, \dots, K-1$, the right side telescopes:

$$\sum_{k=1}^K (f(\mathbf{x}_k) - f^\star) \leq \frac{L}{2} \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

The values $f(\mathbf{x}_k)$ never increase, so the left side is at least $K(f(\mathbf{x}_K) - f^\star)$. ■

Strongly convex functions: the error shrinks geometrically

Theorem (L -smooth, μ -strongly convex, $\alpha = 1/L$)

$$\|\mathbf{x}_k - \mathbf{x}^\star\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2 = \left(1 - \frac{1}{\kappa}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

Proof. Write $\mathbf{e} = \mathbf{x}_k - \mathbf{x}^\star$, $\Delta = f(\mathbf{x}_k) - f^\star$, and expand $\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 = \|\mathbf{e}\|^2 - \frac{2}{L} \mathbf{g}_k^\top \mathbf{e} + \frac{1}{L^2} \|\mathbf{g}_k\|^2$. Two facts bound the middle and last terms:

Strongly convex functions: the error shrinks geometrically

Theorem (L -smooth, μ -strongly convex, $\alpha = 1/L$)

$$\|\mathbf{x}_k - \mathbf{x}^\star\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2 = \left(1 - \frac{1}{\kappa}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

Proof. Write $\mathbf{e} = \mathbf{x}_k - \mathbf{x}^\star$, $\Delta = f(\mathbf{x}_k) - f^\star$, and expand $\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 = \|\mathbf{e}\|^2 - \frac{2}{L} \mathbf{g}_k^\top \mathbf{e} + \frac{1}{L^2} \|\mathbf{g}_k\|^2$. Two facts bound the middle and last terms:

- strong convexity at $\mathbf{y} = \mathbf{x}^\star$: $\mathbf{g}_k^\top \mathbf{e} \geq \Delta + \frac{\mu}{2} \|\mathbf{e}\|^2$;
- the descent lemma at $\mathbf{x}_k - \mathbf{g}_k/L$: $f^\star \leq f(\mathbf{x}_k) - \frac{1}{2L} \|\mathbf{g}_k\|^2$, i.e. $\|\mathbf{g}_k\|^2 \leq 2L\Delta$.

Strongly convex functions: the error shrinks geometrically

Theorem (L -smooth, μ -strongly convex, $\alpha = 1/L$)

$$\|\mathbf{x}_k - \mathbf{x}^\star\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2 = \left(1 - \frac{1}{\kappa}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

Proof. Write $\mathbf{e} = \mathbf{x}_k - \mathbf{x}^\star$, $\Delta = f(\mathbf{x}_k) - f^\star$, and expand $\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 = \|\mathbf{e}\|^2 - \frac{2}{L} \mathbf{g}_k^\top \mathbf{e} + \frac{1}{L^2} \|\mathbf{g}_k\|^2$. Two facts bound the middle and last terms:

- strong convexity at $\mathbf{y} = \mathbf{x}^\star$: $\mathbf{g}_k^\top \mathbf{e} \geq \Delta + \frac{\mu}{2} \|\mathbf{e}\|^2$;
- the descent lemma at $\mathbf{x}_k - \mathbf{g}_k/L$: $f^\star \leq f(\mathbf{x}_k) - \frac{1}{2L} \|\mathbf{g}_k\|^2$, i.e. $\|\mathbf{g}_k\|^2 \leq 2L\Delta$.

$$\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 \leq \|\mathbf{e}\|^2 - \frac{2}{L} \Delta - \frac{\mu}{L} \|\mathbf{e}\|^2 + \frac{2}{L} \Delta = \left(1 - \frac{\mu}{L}\right) \|\mathbf{e}\|^2. \quad \blacksquare$$

Strongly convex functions: the error shrinks geometrically

Theorem (L -smooth, μ -strongly convex, $\alpha = 1/L$)

$$\|\mathbf{x}_k - \mathbf{x}^\star\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2 = \left(1 - \frac{1}{\kappa}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|^2.$$

Proof. Write $\mathbf{e} = \mathbf{x}_k - \mathbf{x}^\star$, $\Delta = f(\mathbf{x}_k) - f^\star$, and expand $\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 = \|\mathbf{e}\|^2 - \frac{2}{L} \mathbf{g}_k^\top \mathbf{e} + \frac{1}{L^2} \|\mathbf{g}_k\|^2$. Two facts bound the middle and last terms:

- strong convexity at $\mathbf{y} = \mathbf{x}^\star$: $\mathbf{g}_k^\top \mathbf{e} \geq \Delta + \frac{\mu}{2} \|\mathbf{e}\|^2$;
- the descent lemma at $\mathbf{x}_k - \mathbf{g}_k/L$: $f^\star \leq f(\mathbf{x}_k) - \frac{1}{2L} \|\mathbf{g}_k\|^2$, i.e. $\|\mathbf{g}_k\|^2 \leq 2L\Delta$.

$$\|\mathbf{x}_{k+1} - \mathbf{x}^\star\|^2 \leq \|\mathbf{e}\|^2 - \frac{2}{L} \Delta - \frac{\mu}{L} \|\mathbf{e}\|^2 + \frac{2}{L} \Delta = \left(1 - \frac{\mu}{L}\right) \|\mathbf{e}\|^2. \quad \blacksquare$$

With $\alpha = \frac{2}{\mu+L}$ the factor improves to $\|\mathbf{e}_{k+1}\| \leq \frac{\kappa-1}{\kappa+1} \|\mathbf{e}_k\|$, the same number the quadratic analysis gave in Section 5 [15].

Linear rates without convexity: the Polyak–Łojasiewicz condition

PL inequality [11]

$\frac{1}{2} \|\nabla f(\mathbf{x})\|^2 \geq \mu (f(\mathbf{x}) - f^\star)$ for all $\mathbf{x}$. Then GD with $\alpha = 1/L$ satisfies

$$f(\mathbf{x}_k) - f^\star \leq \left(1 - \frac{\mu}{L}\right)^k (f(\mathbf{x}_0) - f^\star).$$

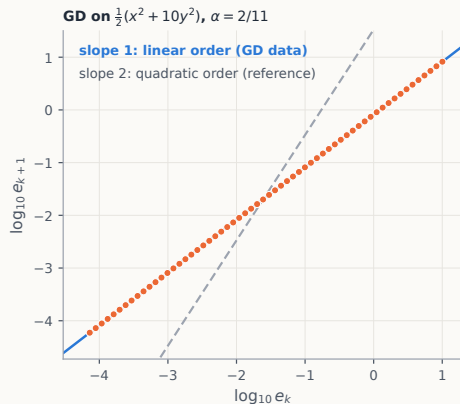
Proof (two lines). $f(\mathbf{x}_{k+1}) - f^\star \leq f(\mathbf{x}_k) - f^\star - \frac{1}{2L} \|\mathbf{g}_k\|^2 \leq \left(1 - \frac{\mu}{L}\right)(f(\mathbf{x}_k) - f^\star)$. ■

- PL does *not* need convexity. $f(x) = x^2 + 3 \sin^2 x$ has f'' dipping to -4 , yet it satisfies PL (numerically $\mu \approx 0.175$).
- Strong convexity implies PL (from the Section 6 sandwich), so PL is the more general route to a linear rate.
- Least squares $\frac{1}{2} \|\mathbf{M}\mathbf{x} - \mathbf{b}\|^2$ with a rank-deficient $\mathbf{M}$ is convex but not strongly convex. It still satisfies PL, so GD still converges linearly in f [11].

A sequence $\{\mathbf{e}_k\}$ has **order** p if $\|\mathbf{e}_{k+1}\| \approx r \|\mathbf{e}_k\|^p$.

- GD has $p = 1$, **linear** order, with $r \leq \rho$. It gains the *same* number of correct digits, $-\log_{10} \rho$, every step.
- In the plot, $\log e_{k+1}$ against $\log e_k$ is a line of slope 1. The dashed slope-2 line (quadratic order, as in Newton-type methods) is there for comparison.

Setting	Guarantee ($\alpha = \frac{1}{L}$)	Iterations
nonconvex	$\min \ \mathbf{g}\ ^2 \leq \frac{2L\Delta_0}{K}$	$O(\varepsilon^{-2})$
convex	$f - f^\star \leq \frac{LR^2}{2k}$	$O(\varepsilon^{-1})$
strongly convex	$\ \mathbf{e}_k\ ^2 \leq (1 - \frac{1}{\kappa})^k R^2$	$O(\kappa \log \frac{1}{\varepsilon})$
PL	$f - f^\star \leq (1 - \frac{1}{\kappa})^k \Delta_0$	$O(\kappa \log \frac{1}{\varepsilon})$



$$R = \|\mathbf{x}_0 - \mathbf{x}^\star\| \text{ and } \Delta_0 = f(\mathbf{x}_0) - f^\star.$$

When GD, and when Newton? Count the work for n parameters [17].

	Gradient descent	Newton's method
information used	gradient (n numbers)	gradient + Hessian (n^2 numbers)
cost of one step	$O(n)$ after the gradient	$O(n^3)$ to solve $H\mathbf{y} = \mathbf{g}$
order of convergence	linear	quadratic
learning rate	must be chosen	not needed
on a concave stretch ($f'' < 0$)	still goes downhill	can head for a maximum
best for	large problems (ML, $n \sim 10^6$)	small problems needing high accuracy

Why ML uses GD

With $n = 10^6$ a Hessian has 10^{12} entries and will not fit in memory; GD stores n numbers. When H is not positive definite, Levenberg–Marquardt [17] moves towards a gradient step.

08

Why it can be so slow: conditioning



Exact line search takes the *best* step along $-\mathbf{g}_k$: $\alpha_k = \arg \min_{\alpha > 0} f(\mathbf{x}_k - \alpha \mathbf{g}_k)$.

- For a quadratic there is a closed form, $\alpha_k = \frac{\mathbf{g}_k^T \mathbf{g}_k}{\mathbf{g}_k^T \mathbf{A} \mathbf{g}_k}$, at the price of one extra matrix-vector product.
- For general f it is a 1-D minimisation every iteration. Any 1-D method works (golden-section search, for example, which is a separate topic). In practice cheaper inexact rules win (Section 10).

Consecutive steps are perpendicular

At the optimal α_k , $0 = \frac{d}{d\alpha} f(\mathbf{x}_k - \alpha \mathbf{g}_k) \big|_{\alpha_k} = -\nabla f(\mathbf{x}_{k+1})^T \mathbf{g}_k$, so $\mathbf{g}_{k+1} \perp \mathbf{g}_k$.

So every step turns exactly 90° . In a long narrow valley that produces the famous **zig-zag** (Problem M4 proves it and gives the iterates in closed form).

Theorem (steepest descent with exact line search, quadratic f)

$$f(\mathbf{x}_{k+1}) - f^* \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 (f(\mathbf{x}_k) - f^*).$$

Proof.

Theorem (steepest descent with exact line search, quadratic f)

$$f(\mathbf{x}_{k+1}) - f^* \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 (f(\mathbf{x}_k) - f^*).$$

Proof. Shift so that $f - f^* = \frac{1}{2} \mathbf{e}^T \mathbf{A} \mathbf{e} = \frac{1}{2} \mathbf{g}^T \mathbf{A}^{-1} \mathbf{g}$ with $\mathbf{g} = \mathbf{A} \mathbf{e}$. The exact step gives $f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \frac{(\mathbf{g}^T \mathbf{g})^2}{2 \mathbf{g}^T \mathbf{A} \mathbf{g}}$, hence

$$\frac{f(\mathbf{x}_{k+1}) - f^*}{f(\mathbf{x}_k) - f^*} = 1 - \frac{(\mathbf{g}^T \mathbf{g})^2}{(\mathbf{g}^T \mathbf{A} \mathbf{g})(\mathbf{g}^T \mathbf{A}^{-1} \mathbf{g})}.$$

Theorem (steepest descent with exact line search, quadratic f)

$$f(\mathbf{x}_{k+1}) - f^* \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 (f(\mathbf{x}_k) - f^*).$$

Proof. Shift so that $f - f^* = \frac{1}{2} \mathbf{e}^T \mathbf{A} \mathbf{e} = \frac{1}{2} \mathbf{g}^T \mathbf{A}^{-1} \mathbf{g}$ with $\mathbf{g} = \mathbf{A} \mathbf{e}$. The exact step gives $f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \frac{(\mathbf{g}^T \mathbf{g})^2}{2 \mathbf{g}^T \mathbf{A} \mathbf{g}}$, hence

$$\frac{f(\mathbf{x}_{k+1}) - f^*}{f(\mathbf{x}_k) - f^*} = 1 - \frac{(\mathbf{g}^T \mathbf{g})^2}{(\mathbf{g}^T \mathbf{A} \mathbf{g})(\mathbf{g}^T \mathbf{A}^{-1} \mathbf{g})}.$$

The **Kantorovich inequality** says the fraction is at least $\frac{4\lambda_{\min}\lambda_{\max}}{(\lambda_{\min} + \lambda_{\max})^2}$, so the ratio is at most $1 - \frac{4\kappa}{(1+\kappa)^2} = \left(\frac{\kappa-1}{\kappa+1} \right)^2$. ■

Theorem (steepest descent with exact line search, quadratic f)

$$f(\mathbf{x}_{k+1}) - f^* \leq \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 (f(\mathbf{x}_k) - f^*).$$

Proof. Shift so that $f - f^* = \frac{1}{2} \mathbf{e}^T \mathbf{A} \mathbf{e} = \frac{1}{2} \mathbf{g}^T \mathbf{A}^{-1} \mathbf{g}$ with $\mathbf{g} = \mathbf{A} \mathbf{e}$. The exact step gives $f(\mathbf{x}_{k+1}) = f(\mathbf{x}_k) - \frac{(\mathbf{g}^T \mathbf{g})^2}{2 \mathbf{g}^T \mathbf{A} \mathbf{g}}$, hence

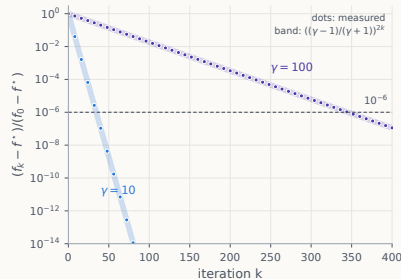
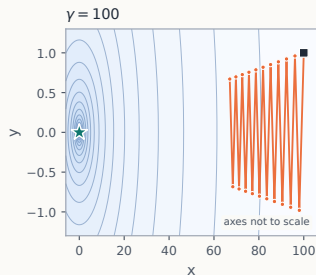
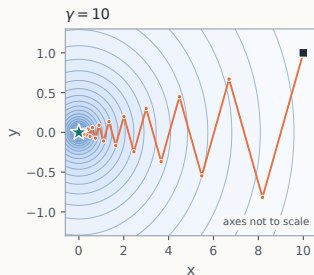
$$\frac{f(\mathbf{x}_{k+1}) - f^*}{f(\mathbf{x}_k) - f^*} = 1 - \frac{(\mathbf{g}^T \mathbf{g})^2}{(\mathbf{g}^T \mathbf{A} \mathbf{g})(\mathbf{g}^T \mathbf{A}^{-1} \mathbf{g})}.$$

The **Kantorovich inequality** says the fraction is at least $\frac{4\lambda_{\min}\lambda_{\max}}{(\lambda_{\min} + \lambda_{\max})^2}$, so the ratio is at most $1 - \frac{4\kappa}{(1+\kappa)^2} = \left(\frac{\kappa-1}{\kappa+1} \right)^2$. ■

And the bound is **sharp**. Problem M4 finds a starting point that hits it exactly, at every single step [17, 4].

The zig-zag in pictures

Why it can be so slow: conditioning



- On $f = \frac{1}{2}(x^2 + \gamma y^2)$ from $(\gamma, 1)$, cutting f by 10^6 takes **35** steps when $\gamma = 10$ and **346** when $\gamma = 100$. That is about 3.45κ .
- A perfect *step length* cannot rescue a bad *direction*. There are two ways out: change the geometry (preconditioning, next slide) or remember where you have been (momentum, Section 11).

The fix: change the geometry (preconditioning)

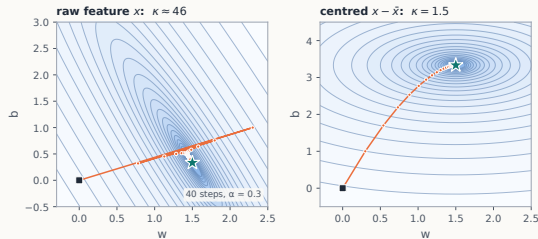
Why it can be so slow: conditioning

Run GD on $\tilde{f}(\mathbf{y}) = f(\mathbf{P}^{-1/2}\mathbf{y})$. In the original variables that is

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{P}^{-1} \nabla f(\mathbf{x}_k),$$

with Hessian $\mathbf{P}^{-1/2} \mathbf{A} \mathbf{P}^{-1/2}$. Pick $\mathbf{P} \approx \mathbf{A}$ and $\tilde{\kappa} \approx 1$.

- **Centring features** (A1): κ from 46.1 to 1.5, iterations from 319 to 9.
- **Jacobi scaling** $\mathbf{P} = \text{diag}(\mathbf{A})$ (A2): κ from 1002 to 1.065.
- $\mathbf{P} = \nabla^2 f$ is ideal but costly: Newton's method, left aside here.



The A1 regression before and after centring. The valley turns into a bowl.

WHY IT IS SLOW

**The step length is not the problem.
The direction is.**

Fix the geometry (preconditioning) or remember the past (momentum).

09

When gradient descent goes wrong

start ● ● ● ● ● ● ○ ○ ○ ○ ○ ○ Part B

Failure modes

1. Step too large ($\alpha > 2/L$): oscillating blow-up along the top eigenvector (M3(f), A3(d)).

2. No global L . For $f(x) = x^4$, $f'' = 12x^2$ keeps growing. With $\alpha = 0.1$,

$$x_{k+1} = x_k (1 - 0.4x_k^2),$$

which diverges exactly when

$$|x_0| > \sqrt{5} \approx 2.236:$$

$$x_0 = 2.2: \quad -2.059, 1.434, 0.255, 0.249, \dots \rightarrow 0$$

$$x_0 = 2.3: \quad -2.567, 4.198, -25.39, 6521, \dots$$

3. A flat minimum. At $x^* = 0$ we have $f''(0) = 0$, so f is not strongly convex and the rate drops to *sublinear*:

$$x_{k+1} \approx x_k - 0.4x_k^3 \Rightarrow x_k \approx \frac{1}{\sqrt{0.8k}}.$$

Check: at $k = 10^4$ the iteration gives 0.011176 and the formula 0.011180.

Lesson

A safe step depends on *where you are*. A fixed α is only safe with a global L ; otherwise let a line search pick the step (Section 10).

Saddle points: a trap with a trapdoor

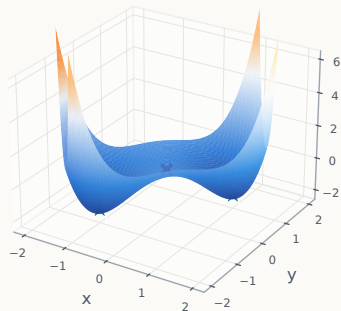
When gradient descent goes wrong

$f = x^4 + y^4 - 4xy$ has a saddle at 0, with Hessian eigenvalues -4 along $(1, 1)$ and $+4$ along $(1, -1)$.

- On the **stable manifold** $y = -x$, GD slides *into* the saddle (M1).
- Elsewhere the unstable part grows by $1 + 4\alpha$ per step: escaping from distance $\delta = 10^{-3}$ takes about 21 steps ($\alpha = 0.1$).

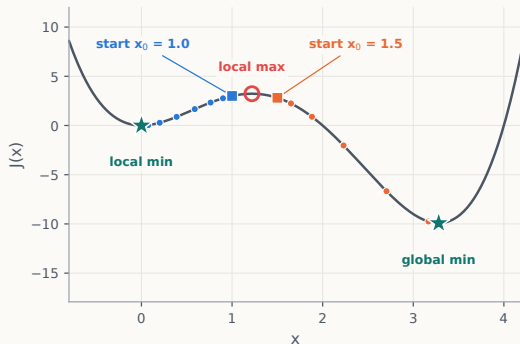
Theorem [13]

For C^2 f with L -Lipschitz gradient and $\alpha < 1/L$, GD from a random start converges to a strict saddle with probability zero.



GD almost never *ends* at a saddle, but can idle near one: $\|\nabla f\|$ is tiny there.

Local minima: the starting point decides the answer



$f(x) = x^4 - 6x^3 + 8x^2$ has two valleys with a hill at $x \approx 1.22$.

- From $x_0 = 1.0$ GD rolls left into the *local* minimum $x = 0$.
- From $x_0 = 1.5$ it rolls right into the *global* minimum $x \approx 3.28$.

GD may find *a* minimum and miss *the* best one.

In practice

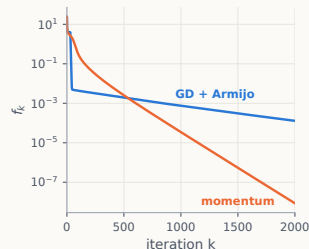
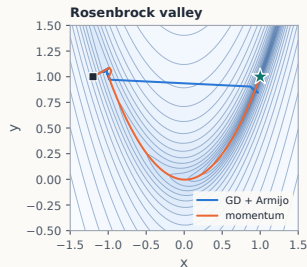
Try several starts and keep the best. The SSR of a line is convex: one minimum only.

Plateau (A4). For large k the model e^{-kt} is already close to zero, so it barely reacts to k and $J' \approx 10^{-3}$. A thousand steps from $k_0 = 6$ only reach $k \approx 4.65$: a **vanishing gradient**.

The banana valley (Rosenbrock)

$$f = (1 - x)^2 + 100(y - x^2)^2$$

has $\kappa \approx 2508$ at the minimum, and the valley *bends*, so no single fixed rescaling straightens it. On the right: GD with Armijo against momentum over 2000 iterations.



The A4 plateau itself is plotted in Problem A4 (solution 3 of 4).

Nonsmooth objectives: a constant step never settles

$f(x) = |x|$ has “gradient” $\text{sign}(x)$, whose size never shrinks near the minimum:

$$x_{k+1} = x_k - \alpha \text{sign}(x_k).$$

- The iterates march into the band $|x| \leq \alpha$, then **bounce around in it forever**: the error stays about α .
- In the smooth case $\nabla f \rightarrow 0$ shrinks the steps for free (Section 4). Here it does not.
- Remedy: shrink the steps yourself (the subgradient method), at the slower rate $O(1/\sqrt{k})$ [15, 4].

Where you meet this

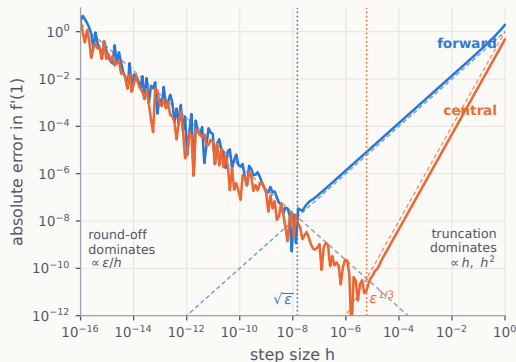
ℓ_1 regularisation, hinge loss, ReLU networks, max-type objectives. Smoothness is what makes a constant step work.

No gradient formula? Try $f'(x) \approx \frac{f(x+h)-f(x)}{h}$,
with error

$$E(h) \approx \underbrace{\frac{h}{2}|f''|}_{\text{truncation}} + \underbrace{\frac{2\epsilon_{\text{mach}}|f|}{h}}_{\text{round-off}},$$

smallest at $h^* \sim \sqrt{\epsilon_{\text{mach}}} \approx 1.5 \times 10^{-8}$. Going
smaller makes it *worse*.

- Forward: about 8 digits at best. Central: $h^* \sim \epsilon^{1/3}$,
error $\sim \epsilon^{2/3} \approx 4 \times 10^{-11}$.
- Cost: n extra f -evaluations per gradient (autodiff:
a constant factor, Section 13).
- $\|\nabla f\|$ has a noise floor: use relative stopping tests.



The classic V: truncation error on the right,
round-off on the left ($f = e^x$ at $x = 1$).

10

Letting the method choose its own step



Accept a step only if it buys a fair share of the decrease the gradient promises. That is the **Armijo condition**, with $c \in (0, 1)$:

$$f(\mathbf{x}_k - \alpha \mathbf{g}_k) \leq f(\mathbf{x}_k) - c \alpha \|\mathbf{g}_k\|^2.$$

Backtracking [1]

$$\alpha \leftarrow \alpha_0$$

while Armijo fails: $\alpha \leftarrow \beta \alpha$ ($0 < \beta < 1$)

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \mathbf{g}_k$$

Common choices: $c = 10^{-4}$, $\beta = \frac{1}{2}$, and $\alpha_0 = 1$ or the previous step.

Theorem: backtracking always stops

If ∇f is L -Lipschitz, Armijo holds for every $\alpha \leq \frac{2(1-c)}{L}$, so the step you accept satisfies

$$\alpha \geq \min \left\{ \alpha_0, \frac{2\beta(1-c)}{L} \right\}.$$

Proof.

Accept a step only if it buys a fair share of the decrease the gradient promises. That is the **Armijo condition**, with $c \in (0, 1)$:

$$f(\mathbf{x}_k - \alpha \mathbf{g}_k) \leq f(\mathbf{x}_k) - c \alpha \|\mathbf{g}_k\|^2.$$

Backtracking [1]

$$\alpha \leftarrow \alpha_0$$

while Armijo fails: $\alpha \leftarrow \beta \alpha \quad (0 < \beta < 1)$

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \mathbf{g}_k$$

Common choices: $c = 10^{-4}$, $\beta = \frac{1}{2}$, and $\alpha_0 = 1$ or the previous step.

Theorem: backtracking always stops

If ∇f is L -Lipschitz, Armijo holds for every $\alpha \leq \frac{2(1-c)}{L}$, so the step you accept satisfies

$$\alpha \geq \min \left\{ \alpha_0, \frac{2\beta(1-c)}{L} \right\}.$$

Proof. The descent lemma gives

$$f(\mathbf{x} - \alpha \mathbf{g}) \leq f(\mathbf{x}) - \alpha \left(1 - \frac{\alpha L}{2}\right) \|\mathbf{g}\|^2, \text{ and}$$
$$1 - \frac{\alpha L}{2} \geq c \iff \alpha \leq \frac{2(1-c)}{L}.$$

Accept a step only if it buys a fair share of the decrease the gradient promises. That is the **Armijo condition**, with $c \in (0, 1)$:

$$f(\mathbf{x}_k - \alpha \mathbf{g}_k) \leq f(\mathbf{x}_k) - c \alpha \|\mathbf{g}_k\|^2.$$

Backtracking [1]

$$\alpha \leftarrow \alpha_0$$

while Armijo fails: $\alpha \leftarrow \beta \alpha \quad (0 < \beta < 1)$

$$\mathbf{x}_{k+1} \leftarrow \mathbf{x}_k - \alpha \mathbf{g}_k$$

Common choices: $c = 10^{-4}$, $\beta = \frac{1}{2}$, and $\alpha_0 = 1$ or the previous step.

Theorem: backtracking always stops

If ∇f is L -Lipschitz, Armijo holds for every $\alpha \leq \frac{2(1-c)}{L}$, so the step you accept satisfies

$$\alpha \geq \min \left\{ \alpha_0, \frac{2\beta(1-c)}{L} \right\}.$$

Proof. The descent lemma gives

$f(\mathbf{x} - \alpha \mathbf{g}) \leq f(\mathbf{x}) - \alpha(1 - \frac{\alpha L}{2}) \|\mathbf{g}\|^2$, and $1 - \frac{\alpha L}{2} \geq c \iff \alpha \leq \frac{2(1-c)}{L}$. Backtracking stops at the first $\alpha_0 \beta^j$ below that threshold, so it is never smaller than $\beta \cdot \frac{2(1-c)}{L}$. ■

- Every accepted step lowers f by at least $c \alpha_{\min} \|\mathbf{g}_k\|^2$, so **all the rates from Section 7 survive**, up to constants, without ever computing L [17].
- The step *adapts to the local curvature*. It shrinks where the curvature is high, which cures the x^4 blow-up from Section 9, and with a generous first trial α_0 it stays long where f is flat.
- The price is a few extra evaluations of f per iteration. No extra gradients are needed.

The Wolfe conditions

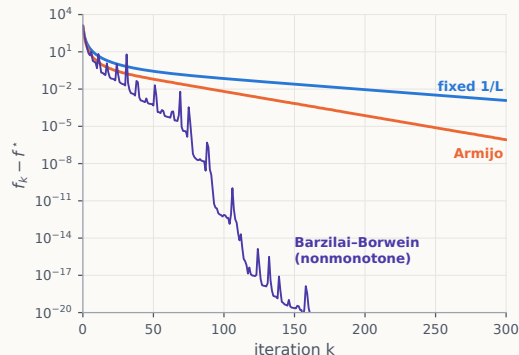
Add a curvature test, $\nabla f(\mathbf{x}_{k+1})^T \mathbf{d}_k \geq c_2 \nabla f(\mathbf{x}_k)^T \mathbf{d}_k$ with $c < c_2 < 1$, and you also rule out steps that are too short. This pair is the standard line search in quasi-Newton software [17].

Barzilai–Borwein: a step size learned from the last move

With $\mathbf{s}_k = \mathbf{x}_k - \mathbf{x}_{k-1}$ and $\mathbf{y}_k = \mathbf{g}_k - \mathbf{g}_{k-1}$, pick the scalar $\beta = \frac{1}{\alpha}$ that best imitates the Hessian, $\beta \mathbf{s}_k \approx \mathbf{y}_k$:

$$\min_{\beta} \|\beta \mathbf{s}_k - \mathbf{y}_k\|^2 \Rightarrow \alpha_k^{\text{BB}} = \frac{\mathbf{s}_k^T \mathbf{s}_k}{\mathbf{s}_k^T \mathbf{y}_k}$$

- On a quadratic, $1/\alpha_k$ is a Rayleigh quotient of $\mathbf{A}$, between $\lambda_{\min}$ and $\lambda_{\max}$.
- **Nonmonotone**: α_k can exceed $2/\lambda_{\max}$ and f can briefly rise, yet it is often far faster [2].
- For general f , add a nonmonotone line-search safeguard.



A diagonal quadratic with $n = 50$ and $\kappa = 100$. Fixed $1/L$ against Armijo against BB.

11

Momentum and acceleration



Polyak's heavy ball [18]: $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k) + \beta (\mathbf{x}_k - \mathbf{x}_{k-1}), \quad 0 \leq \beta < 1.$

Where does that come from? Picture a ball of mass m rolling on the landscape with friction γ :
 $m\ddot{\mathbf{x}} + \gamma\dot{\mathbf{x}} = -\nabla f(\mathbf{x})$. Discretise with step h (central difference for $\ddot{\mathbf{x}}$, forward for $\dot{\mathbf{x}}$):

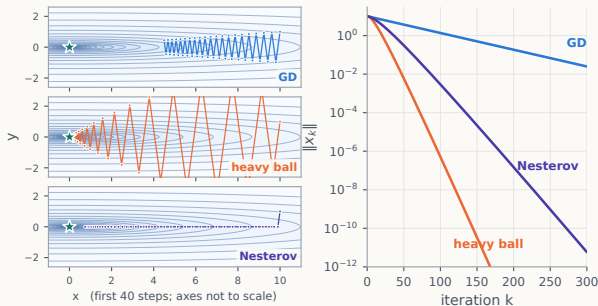
$$m \frac{\mathbf{x}_{k+1} - 2\mathbf{x}_k + \mathbf{x}_{k-1}}{h^2} + \gamma \frac{\mathbf{x}_{k+1} - \mathbf{x}_k}{h} = -\nabla f(\mathbf{x}_k)$$
$$\implies \mathbf{x}_{k+1} = \mathbf{x}_k - \underbrace{\frac{h^2}{m+\gamma h}}_{\alpha} \nabla f(\mathbf{x}_k) + \underbrace{\frac{m}{m+\gamma h}}_{\beta} (\mathbf{x}_k - \mathbf{x}_{k-1}).$$

- The velocity $\mathbf{x}_k - \mathbf{x}_{k-1}$ **builds up** where the gradient keeps its sign (down the valley) and **cancels** where it flips (across it), so the zig-zag is damped.
- Plain GD is the massless ball: $m \rightarrow 0$, i.e. $\beta = 0$.

Each eigen-direction obeys
 $r^2 - (1 + \beta - \alpha\lambda)r + \beta = 0$ (M5). Tuned
well, every root sits on $|r| = \sqrt{\beta}$:

$$\rho_{\text{HB}} = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} \text{ vs. } \rho_{\text{GD}} = \frac{\kappa-1}{\kappa+1}.$$

- $\kappa = 100$: 0.818 against 0.980, so 69 iterations instead of 691.
- In general $O(\sqrt{\kappa})$ instead of $O(\kappa)$.
- **Caveat:** proven for quadratics only; counterexamples exist beyond [14].



$f = \frac{1}{2}(x^2 + 100y^2)$: the first 40 steps of each method (left)
and $\|x_k\|$ (right).

The trick is to **take the gradient at a look-ahead point** [16]:

$$\mathbf{y}_k = \mathbf{x}_k + \beta_k(\mathbf{x}_k - \mathbf{x}_{k-1}), \quad \mathbf{x}_{k+1} = \mathbf{y}_k - \frac{1}{L} \nabla f(\mathbf{y}_k).$$

Convex, L -smooth, $\beta_k = \frac{k-1}{k+2}$

$$f(\mathbf{x}_k) - f^* \leq \frac{2L \|\mathbf{x}_0 - \mathbf{x}^*\|^2}{(k+1)^2} \quad \text{vs. GD: } \frac{L \|\mathbf{x}_0 - \mathbf{x}^*\|^2}{2k}.$$

μ -strongly convex, $\beta = \frac{\sqrt{k}-1}{\sqrt{k}+1}$

$$f(\mathbf{x}_k) - f^* \leq \left(1 - \frac{1}{\sqrt{k}}\right)^k \left(f(\mathbf{x}_0) - f^* + \frac{\mu}{2} \|\mathbf{x}_0 - \mathbf{x}^*\|^2\right).$$

GD's bound after 10^4 steps, Nesterov reaches after about 200, for *every* smooth convex function [15].

Continuous-time limit: $\ddot{\mathbf{x}} + \frac{3}{t} \dot{\mathbf{x}} + \nabla f = 0$ [20], a heavy ball with fading friction.

Lower bounds [15]

For any method with iterates in $\mathbf{x}_0 + \text{span}\{\nabla f(\mathbf{x}_0), \dots, \nabla f(\mathbf{x}_{k-1})\}$ (Nesterov matches both up to a constant):

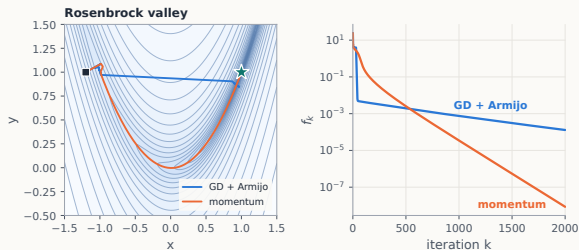
- convex, L -smooth (high enough dimension): $f(\mathbf{x}_k) - f^\star \geq \frac{3L\|\mathbf{x}_0 - \mathbf{x}^\star\|^2}{32(k+1)^2}$;
- strongly convex: $\|\mathbf{x}_k - \mathbf{x}^\star\| \geq \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^k \|\mathbf{x}_0 - \mathbf{x}^\star\|$.

	GD	Heavy ball	Nesterov
convex rate	$O(1/k)$	no guarantee	$O(1/k^2)$ (optimal)
strongly convex	$O(\kappa \log \frac{1}{\epsilon})$	$O(\sqrt{\kappa} \log \frac{1}{\epsilon})$ on quadratics	$O(\sqrt{\kappa} \log \frac{1}{\epsilon})$ (optimal)
$f(\mathbf{x}_k)$ always decreases?	yes ($\alpha \leq \frac{1}{L}$)	no	no ("ripples")

THE PRICE OF A PROBLEM

**The condition number κ
decides how many steps you pay.**

GD pays $O(\kappa)$. Nesterov pays $O(\sqrt{\kappa})$, and nothing that only sees gradients can do better.



- On Rosenbrock, after 2000 iterations, momentum ($\beta = 0.9$) is down to $f \approx 10^{-8}$ while GD with Armijo is stuck near 10^{-4} .
- Momentum methods are **non-monotone**. f ripples up and down as the ball overshoots and swings back.
- A common practical fix: *restart* the momentum whenever f goes up.
- Momentum is built into the optimisers people actually use, such as SGD with momentum and Adam (Section 13).

For more intuition, see [9].

12

Stochastic gradient descent

start ● ● ● ● ● ● ● ● ● ● ○ ○ Part B

SGD and modern

A training loss averages over N data points:

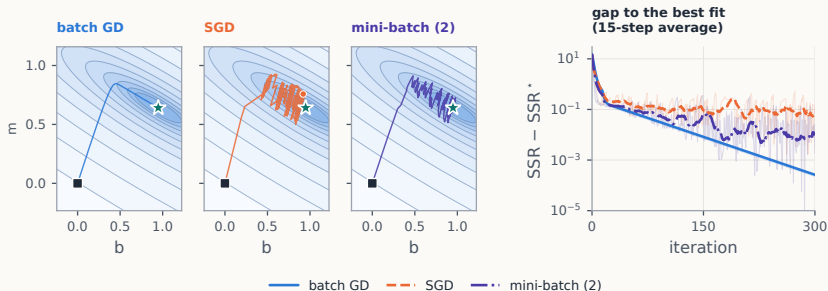
$$f(\mathbf{x}) = \frac{1}{N} \sum_{i=1}^N f_i(\mathbf{x}), \quad \text{so one } \nabla f \text{ costs } N \text{ component gradients.}$$

SGD cheats: pick one i_k at random and step $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \nabla f_{i_k}(\mathbf{x}_k)$.

- It is **unbiased**: $\mathbb{E}_i[\nabla f_i(\mathbf{x})] = \nabla f(\mathbf{x})$, so SGD is gradient descent *on average*.
- A step costs the same whatever N is. With $N = 10^6$, one GD step buys a million SGD steps.
- A **mini-batch** of B samples cuts the gradient variance to σ^2/B , at B times the cost.

The catch

The noise does *not* switch off at the answer: $\nabla f(\mathbf{x}^*) = 0$, but $\nabla f_i(\mathbf{x}^*) \neq 0$. With a constant step, SGD keeps jittering and never settles exactly.



- One-point gradients are *unbiased*: on average they equal the full gradient.
- Their noise does not vanish at the minimum, so with a fixed α SGD keeps jittering. Averaging B points cuts the variance by B ; a decaying α lets the path settle.

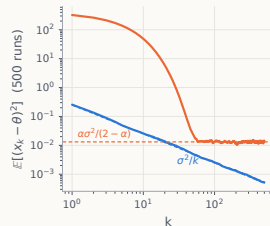
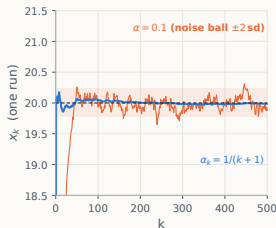
Solvable case (A5): $F = \frac{1}{2}\mathbb{E}(x - \xi)^2$, constant α :

$$\text{Var}(x_k) \rightarrow \frac{\alpha\sigma^2}{2-\alpha}.$$

Any strongly convex f [3]:

- fixed α : linear drop, but only to a floor $\propto \alpha$;
- $\alpha_k = O(1/k)$: $\mathbb{E}[f(\mathbf{x}_k) - f^\star] = O(1/k)$ all the way.

Robbins–Monro [19]: $\sum_k \alpha_k = \infty$ (travel far enough) and $\sum_k \alpha_k^2 < \infty$ (noise averages out).



With constant $\alpha = 0.1$ the error stalls at $\frac{\alpha\sigma^2}{2-\alpha}$; with $\alpha_k = \frac{1}{k+1}$ it keeps falling like σ^2/k (averaged over 500 runs).

- **Schedules.** Start with a large constant step for fast progress, then decay it to shrink the noise ball. Warm-up and step decay are the usual heuristics.
- **Epochs and shuffling.** Sweep through the data in a random order instead of sampling with replacement.
- **Batch size against step size.** A bigger B gives a cleaner gradient but fewer steps for the same compute.
- **Total work** to reach accuracy ε on a strongly convex problem, roughly:

Work per iteration		Iterations
GD	N	$O(\kappa \log \frac{1}{\varepsilon})$
SGD	1	$O(1/\varepsilon)$ (constants depend on κ and the noise)

Huge N and moderate accuracy (the usual ML situation): SGD's N -free step cost wins [3].

13

Gradient methods today

start ● ● ● ● ● ● ● ● ● ● ○ ○ Part B

SGD and modern

Cheap gradient principle [10]

Reverse-mode automatic differentiation computes the *whole* gradient $\nabla f \in \mathbb{R}^n$ for a small constant multiple of the cost of evaluating f once, **no matter how large n is**.

How you get ∇f	Cost	Accuracy
finite differences (Section 9)	$(n + 1) \times \text{cost}(f)$	$\sim \sqrt{\epsilon_{\text{mach}}}$
forward-mode AD	$\sim n \times \text{cost}(f)$	machine precision
reverse-mode AD (backpropagation)	$O(1) \times \text{cost}(f)$	machine precision

- Backpropagation in neural networks *is* reverse-mode AD. That is why the gradient of a 10^9 -parameter model costs about as much as a few forward passes.
- GD itself only needs matrix–vector style operations, which run in parallel on GPUs.

Adam [12], with all operations element-wise:

$$\mathbf{m}_k = \beta_1 \mathbf{m}_{k-1} + (1 - \beta_1) \mathbf{g}_k$$

$$\mathbf{v}_k = \beta_2 \mathbf{v}_{k-1} + (1 - \beta_2) \mathbf{g}_k^2$$

$$\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \hat{\mathbf{m}}_k / (\sqrt{\hat{\mathbf{v}}_k} + \epsilon)$$

Read it as **momentum** plus a **diagonal preconditioner** $\mathbf{P}_k = \text{diag}(\sqrt{\hat{\mathbf{v}}_k})$. In the language of Section 2, it is steepest descent in a diagonal norm that keeps changing. (The hats are a bias correction.)

The edge of stability [8]

- Classical theory says to keep $\alpha < 2/L$.
- Yet in full-batch GD on neural networks, the *sharpness* $\lambda_{\max}(\nabla^2 f)$ is observed to climb until it reaches $2/\alpha$ and then hover just above it. The loss keeps going down over long horizons, though not monotonically.
- This is an *empirical* finding that the classical analysis of Section 5 to Section 7 does not explain. It is an open research question.

14

Summary

start ● ● ● ● ● ● ● ● ● ● ○ ○ Part B

Part A recap

Method	Step rule	Rate (str. convex)	Cost/iter.	Watch out for
GD, fixed α	$\mathbf{x} - \alpha \nabla f, \alpha < 2/L$	$\frac{\kappa-1}{\kappa+1}; O(\kappa \log \frac{1}{\varepsilon})$	1 grad	divergence, zig-zag
GD + exact LS	$\arg \min_{\alpha} f(\mathbf{x} - \alpha \mathbf{g})$	$(\frac{\kappa-1}{\kappa+1})^2$ in f	1 grad + 1-D	zig-zag
GD + Armijo	backtracking	as GD, no L needed	1 grad + few f	plateaus
Barzilai–Borwein	$\mathbf{s}^T \mathbf{s} / \mathbf{s}^T \mathbf{y}$	fast in practice	1 grad	nonmonotone
Heavy ball	$+\beta(\mathbf{x}_k - \mathbf{x}_{k-1})$	$\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}$ (quadratics)	1 grad	fails off quadratics
Nesterov	grad. at look-ahead	$O(\sqrt{\kappa} \log \frac{1}{\varepsilon})$ (optimal)	1 grad	ripples
SGD	$\mathbf{x} - \alpha_k \nabla f_{i_k}$	$O(1/k), \alpha_k \propto \frac{1}{k}$	$\frac{1}{N}$ grad	noise ball

1. **Gradient descent is forward Euler on the gradient flow.** The step limit $\alpha < 2/L$ is Euler's stability interval, and ill-conditioning is stiffness under another name.
2. **The condition number κ sets the price.** GD needs $O(\kappa)$ iterations; accelerated methods need $O(\sqrt{\kappa})$, and no first-order method can do better.
3. **One inequality powers the proofs.** The descent lemma, paired with convexity, strong convexity or PL, gives every rate in this module.
4. **Failures are predictable.** Divergence, saddles, plateaus, nonsmoothness and round-off each leave a recognisable signature, and each has a known remedy.
5. **Modern practice is the same idea at scale.** SGD, momentum, Adam and autodiff all build on the plain update $\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f$.

IN ONE SENTENCE

**A gradient step is cheap.
The number of steps is the hard part.**

And the geometry of f decides that number, which is what most of the numerical analysis is about.

B

Part B: problems and complete solutions



Every problem is followed **right away** by its full worked solution. Try each one yourself before you look at the next slide.

Mathematical

- M1** Stationary points of a quartic landscape
- M2** The descent lemma and the nonconvex rate
- M3** Step-size stability on a coupled quadratic
- M4** Exact line search and the zig-zag
- M5** Heavy-ball momentum against plain GD

Real-life applications

- A1** Machine learning: regression and feature scaling
- A2** Structural mechanics: a spring chain at rest
- A3** Economics: two-product profit maximisation
- A4** Parameter estimation: Newton's law of cooling
- A5** Signal processing: calibrating a sensor with SGD

Difficulty: ★ routine ★★ several steps ★★★ a proof or a deeper idea. Every number in the solutions was checked numerically.

Problem M1 · Stationary points of a quartic landscape

Problem M1

Mathematical ★★

Consider $f(x, y) = x^4 + y^4 - 4xy$ on $\mathbb{R}^2$.

- (a) Compute ∇f and find *all* stationary points.
- (b) Classify each stationary point using the Hessian.
- (c) Take one gradient-descent step from $\mathbf{x}_0 = (1, 0)$ with $\alpha = 0.1$, and check that f goes down.
- (d) Near the minimisers, which fixed step sizes keep GD locally stable? What happens if GD starts exactly at the saddle, or anywhere on the line $y = -x$?

What this practises: computing gradients, finding and classifying stationary points, and local step-size stability.

(a) Gradient and stationary points.

$$\nabla f(x, y) = \begin{pmatrix} 4x^3 - 4y \\ 4y^3 - 4x \end{pmatrix} = 0 \iff y = x^3, \quad x = y^3.$$

Substitute the first equation into the second:

$$x = (x^3)^3 = x^9 \implies x(x^8 - 1) = 0.$$

Over $\mathbb{R}$, $x^8 = 1$ has only the roots $x = \pm 1$. So $x \in \{-1, 0, 1\}$, and with $y = x^3$ we get three points:

$$(0, 0), \quad (1, 1), \quad (-1, -1)$$

(b) Classify with the Hessian. $\nabla^2 f = \begin{pmatrix} 12x^2 & -4 \\ -4 & 12y^2 \end{pmatrix}.$

- At $(0, 0)$: $\begin{pmatrix} 0 & -4 \\ -4 & 0 \end{pmatrix}$ has eigenvalues -4 (along $(1, 1)$) and $+4$ (along $(1, -1)$). Indefinite, so this is a **saddle point**, with $f = 0$.
- At $(\pm 1, \pm 1)$: $\begin{pmatrix} 12 & -4 \\ -4 & 12 \end{pmatrix}$ has eigenvalues 8 (along $(1, 1)$) and 16 (along $(1, -1)$). Positive definite, so these are **strict local minima**, with $f = 1 + 1 - 4 = -2$.
- Are they global? Since $4|xy| \leq 2(x^2 + y^2)$, we have $f \geq x^4 + y^4 - 2(x^2 + y^2) \rightarrow \infty$, so f is coercive. Hence $f^* = \boxed{-2}$, reached at both minima.

(c) One GD step from $\mathbf{x}_0 = (1, 0)$ with $\alpha = 0.1$.

$$\mathbf{g}_0 = \nabla f(1, 0) = (4 - 0, 0 - 4) = (4, -4)$$

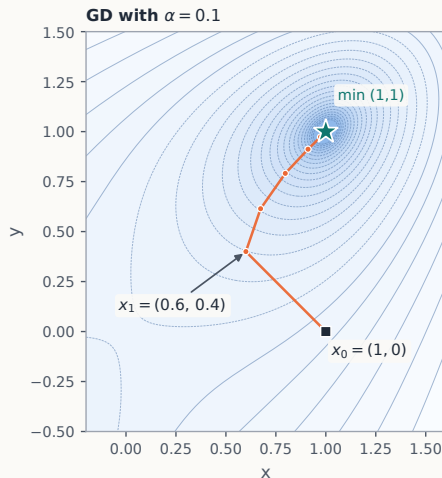
$$\mathbf{x}_1 = (1, 0) - 0.1 (4, -4) = (0.6, 0.4)$$

$$f(\mathbf{x}_0) = 1, \quad f(\mathbf{x}_1) = -0.8048$$

$$(f(\mathbf{x}_1) = 0.6^4 + 0.4^4 - 4(0.24) = 0.1296 + 0.0256 - 0.96.)$$

We knew it would go down for small α , because the directional derivative is

$\nabla f^T(-\mathbf{g}_0) = -\|\mathbf{g}_0\|^2 = -32 < 0$. Keep iterating with $\alpha = 0.1$ and you arrive at $(1, 1)$.



(d) Local stability. Near a minimiser, $\mathbf{e}_{k+1} \approx (\mathbf{I} - \alpha \nabla^2 f(\mathbf{x}^*)) \mathbf{e}_k$, with eigen-factors $1 - 8\alpha$ and $1 - 16\alpha$. The stiffer one decides:

$$|1 - 16\alpha| < 1 \iff 0 < \alpha < \frac{2}{16} = 0.125$$

The best local choice is $\alpha = \frac{2}{8+16} = \frac{1}{12}$, with local rate $\frac{16-8}{16+8} = \frac{1}{3}$. **The saddle.** $\nabla f(0,0) = 0$, so GD started there **never moves at all**. On the line $y = -x$, $\nabla f(t, -t) = (4t^3 + 4t)(1, -1)$ keeps the iterates on the line, and $f(t, -t) = 2t^4 + 4t^2$ is smallest at $t = 0$. So GD **slides into the saddle**: that line is its stable manifold.

Insight

Any nudge along $(1, 1)$ grows by $1 + 4\alpha$ per step and escapes. Numerically, $(10^{-3}, 10^{-3})$ ends at $(1, 1)$, while $(10^{-3}, -10^{-3})$ ends at $(0, 0)$. Saddles only capture a set of starting points of measure zero.

Problem M2

Mathematical ★★★

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be differentiable and bounded below by f^* , with an L -Lipschitz gradient:
 $\|\nabla f(\mathbf{x}) - \nabla f(\mathbf{y})\| \leq L \|\mathbf{x} - \mathbf{y}\|.$

- (a) Prove the *descent lemma*: $f(\mathbf{y}) \leq f(\mathbf{x}) + \nabla f(\mathbf{x})^\top (\mathbf{y} - \mathbf{x}) + \frac{L}{2} \|\mathbf{y} - \mathbf{x}\|^2.$
- (b) Show that GD, $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$, satisfies $f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \alpha \left(1 - \frac{\alpha L}{2}\right) \|\nabla f(\mathbf{x}_k)\|^2.$
- (c) Which α guarantee a decrease? Which α guarantees the largest one?
- (d) Prove $\min_{0 \leq k < K} \|\nabla f(\mathbf{x}_k)\| \leq \sqrt{2L(f(\mathbf{x}_0) - f^*)/K}$ for $\alpha = 1/L.$
- (e) Use $f(x) = \frac{L}{2}x^2$ to show that the bound in (b) cannot be improved.

What this practises: a careful proof, where the $2/L$ step window comes from, and convergence with no convexity at all.

(a) The descent lemma. Let $\mathbf{d} = \mathbf{y} - \mathbf{x}$. By the fundamental theorem of calculus applied to $t \mapsto f(\mathbf{x} + t\mathbf{d})$,

$$f(\mathbf{y}) - f(\mathbf{x}) = \int_0^1 \nabla f(\mathbf{x} + t\mathbf{d})^\top \mathbf{d} \, dt.$$

Subtract $\nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 \nabla f(\mathbf{x})^\top \mathbf{d} \, dt$ from both sides:

$$f(\mathbf{y}) - f(\mathbf{x}) - \nabla f(\mathbf{x})^\top \mathbf{d} = \int_0^1 [\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x})]^\top \mathbf{d} \, dt.$$

Apply Cauchy–Schwarz, then the Lipschitz bound $\|\nabla f(\mathbf{x} + t\mathbf{d}) - \nabla f(\mathbf{x})\| \leq L t \|\mathbf{d}\|$:

$$\leq \int_0^1 L t \|\mathbf{d}\|^2 \, dt = \frac{L}{2} \|\mathbf{d}\|^2. \quad \blacksquare$$

In pictures: f always stays below a parabola of curvature L that touches it at $\mathbf{x}$.

(b) Put $\mathbf{y} = \mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \mathbf{g}_k$ (so $\mathbf{d} = -\alpha \mathbf{g}_k$) into (a):

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \alpha \|\mathbf{g}_k\|^2 + \underbrace{\frac{L\alpha^2}{2} \|\mathbf{g}_k\|^2}_{\varphi(\alpha)} = f(\mathbf{x}_k) - \underbrace{\alpha \left(1 - \frac{\alpha L}{2}\right)}_{\varphi(\alpha)} \|\mathbf{g}_k\|^2.$$

(c) $\varphi(\alpha) > 0 \iff 0 < \alpha < 2/L$. $\varphi'(\alpha) = 1 - \alpha L = 0 \Rightarrow \alpha = 1/L$, which gives $\varphi = \frac{1}{2L}$:

$$f(\mathbf{x}_{k+1}) \leq f(\mathbf{x}_k) - \frac{1}{2L} \|\nabla f(\mathbf{x}_k)\|^2.$$

What this means

The window $(0, 2/L)$ is *exactly* the stability limit we found for quadratics ($\alpha < 2/\lambda_{\max}$). The descent lemma carries it over to every L -smooth function.

(d) Telescoping. Add up the $\alpha = 1/L$ inequality for $k = 0, \dots, K-1$:

$$\frac{1}{2L} \sum_{k=0}^{K-1} \|\mathbf{g}_k\|^2 \leq \sum_{k=0}^{K-1} (f(\mathbf{x}_k) - f(\mathbf{x}_{k+1})) = f(\mathbf{x}_0) - f(\mathbf{x}_K) \leq f(\mathbf{x}_0) - f^*.$$

The smallest term is no bigger than the average, so

$$K \min_{k < K} \|\mathbf{g}_k\|^2 \leq \sum_{k < K} \|\mathbf{g}_k\|^2 \leq 2L(f(\mathbf{x}_0) - f^*) \implies \boxed{\min_{k < K} \|\mathbf{g}_k\| \leq \sqrt{\frac{2L(f(\mathbf{x}_0) - f^*)}{K}}} \quad \blacksquare$$

- The rate is $O(1/\sqrt{K})$: reaching $\|\mathbf{g}\| \leq \varepsilon$ takes $K \geq 2L(f_0 - f^*)/\varepsilon^2$ iterations.
- We never used convexity. The price is that we only get a **small gradient**, and the point we reach could be a saddle (compare M1).

(e) The bound is tight. Take $f(x) = \frac{L}{2}x^2$, so $g = Lx$ and $x^+ = (1 - \alpha L)x$.

$$f(x) - f(x^+) = \frac{L}{2}x^2[1 - (1 - \alpha L)^2] = \frac{L}{2}x^2(2\alpha L - \alpha^2 L^2) = \alpha\left(1 - \frac{\alpha L}{2}\right) \underbrace{L^2 x^2}_{g^2}.$$

The inequality in (b) holds with **equality**, so its constant cannot be improved in general. At $\alpha = 2/L$ we get $x^+ = -x$: the iterate bounces between x and $-x$ forever and f never drops.

A numerical sanity check

For $f(\mathbf{x}) = \log(1 + e^{\mathbf{a}^\top \mathbf{x}}) + \frac{1}{2} \|\mathbf{x}\|^2$ with $L = \|\mathbf{a}\|^2 / 4 + 1 = 2.25$, testing 20 000 random pairs $(\mathbf{x}, \mathbf{y})$ gives $\max[f(\mathbf{y}) - \text{bound}] = -5.6 \times 10^{-4} \leq 0$. The lemma holds every time.

Problem M3 · Step-size stability on a coupled quadratic

Problem M3

Mathematical ★★

Let $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^T\mathbf{A}\mathbf{x} - \mathbf{b}^T\mathbf{x}$ with $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}$, and run GD, $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k)$.

- (a) Find $\mathbf{x}^\star$ and show that $\mathbf{e}_k = \mathbf{x}_k - \mathbf{x}^\star$ obeys $\mathbf{e}_{k+1} = (\mathbf{I} - \alpha\mathbf{A})\mathbf{e}_k$.
- (b) Use the eigenpairs of $\mathbf{A}$ to find *all* α for which GD converges from every $\mathbf{x}_0$.
- (c) Find the α that makes the worst-case contraction factor smallest, and that factor.
- (d) With this α and $\mathbf{x}_0 = (3, 0)$, compute $\mathbf{x}_1, \mathbf{x}_2$ and check that $\|\mathbf{e}_2\| = \|\mathbf{e}_0\|/9$.
- (e) How many iterations guarantee $\|\mathbf{e}_k\| \leq 10^{-6} \|\mathbf{e}_0\|$?
- (f) Describe exactly what happens when $\alpha = 0.6$.

What this practises: an exact stability analysis, the best fixed step, and comparing step choices.

(a) Solve $\nabla f(\mathbf{x}) = \mathbf{Ax} - \mathbf{b} = 0$. Since $\det \mathbf{A} = 8$,

$$\mathbf{x}^* = \mathbf{A}^{-1} \mathbf{b} = \frac{1}{8} \begin{pmatrix} 3 & -1 \\ -1 & 3 \end{pmatrix} \begin{pmatrix} 4 \\ 4 \end{pmatrix} = \boxed{\begin{pmatrix} 1 \\ 1 \end{pmatrix}}.$$

Using $\mathbf{b} = \mathbf{Ax}^*$: $\mathbf{e}_{k+1} = \mathbf{x}_k - \alpha(\mathbf{Ax}_k - \mathbf{Ax}^*) - \mathbf{x}^* = (\mathbf{I} - \alpha\mathbf{A}) \mathbf{e}_k$.

Eigen-decomposition. $\det(\mathbf{A} - \lambda\mathbf{I}) = (3 - \lambda)^2 - 1 = 0$ gives

$$\lambda_1 = 2, \quad \mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, -1)^T; \quad \lambda_2 = 4, \quad \mathbf{v}_2 = \frac{1}{\sqrt{2}}(1, 1)^T.$$

Writing $\mathbf{e}_k = c_1^{(k)} \mathbf{v}_1 + c_2^{(k)} \mathbf{v}_2$ splits the iteration into two separate scalar ones:

$$c_i^{(k)} = (1 - \alpha\lambda_i)^k c_i^{(0)}, \quad i = 1, 2.$$

(b) To converge for every $\mathbf{e}_0$ we need $|1 - \alpha\lambda_i| < 1$ for both i :

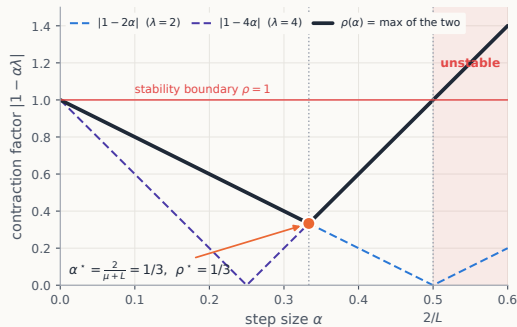
$$0 < \alpha < 1 \quad \text{and} \quad 0 < \alpha < \frac{1}{2}$$

$$\Rightarrow 0 < \alpha < \frac{2}{\lambda_{\max}} = \frac{1}{2}$$

(c) $\rho(\alpha) = \max\{|1 - 2\alpha|, |1 - 4\alpha|\}$ is smallest where the two branches cross:

$$1 - 2\alpha = -(1 - 4\alpha) \Rightarrow \alpha^* = \frac{1}{3}, \quad \rho^* = \frac{1}{3}$$

In general $\alpha^* = \frac{2}{\lambda_{\min} + \lambda_{\max}}$ and $\rho^* = \frac{\kappa - 1}{\kappa + 1}$; here $\kappa = 2$.



(d) With $\alpha = \frac{1}{3}$ and $\mathbf{x}_0 = (3, 0)$:

$$\mathbf{g}_0 = \mathbf{A}\mathbf{x}_0 - \mathbf{b} = (9, 3) - (4, 4) = (5, -1), \quad \mathbf{x}_1 = (3, 0) - \frac{1}{3}(5, -1) = \left(\frac{4}{3}, \frac{1}{3}\right)$$

$$\mathbf{g}_1 = \left(4 + \frac{1}{3}, \frac{4}{3} + 1\right) - (4, 4) = \left(\frac{1}{3}, -\frac{5}{3}\right), \quad \mathbf{x}_2 = \left(\frac{4}{3} - \frac{1}{9}, \frac{1}{3} + \frac{5}{9}\right) = \left(\frac{11}{9}, \frac{8}{9}\right)$$

So $\mathbf{e}_0 = (2, -1)$ and $\mathbf{e}_2 = \left(\frac{2}{9}, -\frac{1}{9}\right) = \frac{1}{9}\mathbf{e}_0$, which gives $\|\mathbf{e}_2\| = \|\mathbf{e}_0\|/9$. ✓

Why exactly 1/9? Split $\mathbf{e}_0 = \frac{1}{2}(1, 1) + \frac{3}{2}(1, -1)$. The factors are $1 - \frac{4}{3} = -\frac{1}{3}$ on $(1, 1)$ and $1 - \frac{2}{3} = \frac{1}{3}$ on $(1, -1)$. Both have size $\frac{1}{3}$, so *every* component shrinks by $\frac{1}{3}$ per step: the optimal step **balances** the two modes.

$$\mathbf{e}_1 = -\frac{1}{6}(1, 1) + \frac{1}{2}(1, -1) = \left(\frac{1}{3}, -\frac{2}{3}\right) \checkmark$$

(e) $\|\mathbf{e}_k\| = 3^{-k} \|\mathbf{e}_0\|$, so we need $3^{-k} \leq 10^{-6}$:

$$k \geq \frac{6 \ln 10}{\ln 3} = \frac{13.8155}{1.0986} = 12.58 \implies k = 13$$

(f) $\alpha = 0.6 > 0.5$. The factors are $1 - 0.6 \cdot 2 = -0.2$ and $1 - 0.6 \cdot 4 = -1.4$:

$$\mathbf{e}_k = \frac{1}{2}(-1.4)^k(1, 1) + \frac{3}{2}(-0.2)^k(1, -1).$$

- The $(1, 1)$ mode **grows** $\times 1.4$ per step, flipping sign each time: $\mathbf{e}_6 \approx (3.765, 3.765)$.
- The $(1, -1)$ mode still decays. One stiff direction ($\lambda_{\max}$) sets the step limit.

Lesson

The blow-up happens along the eigenvector of $\lambda_{\max}$, just as it does for explicit Euler on a stiff ODE.

Problem M4 · Exact line search, the zig-zag, and conditioning

Problem M4

Mathematical ★★★

Steepest descent with *exact line search*: $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha_k \mathbf{g}_k$, $\alpha_k = \arg \min_{\alpha > 0} f(\mathbf{x}_k - \alpha \mathbf{g}_k)$.

- (a) For $f(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x}$ with $\mathbf{A} \succ 0$, show that $\alpha_k = \frac{\mathbf{g}_k^T \mathbf{g}_k}{\mathbf{g}_k^T \mathbf{A} \mathbf{g}_k}$.
- (b) Prove that consecutive gradients are orthogonal: $\mathbf{g}_{k+1}^T \mathbf{g}_k = 0$.
- (c) For $f(x, y) = \frac{1}{2}(x^2 + \gamma y^2)$, $\gamma \geq 1$, $\mathbf{x}_0 = (\gamma, 1)$, prove by induction that $x_k = \gamma r^k$ and $y_k = (-r)^k$, with $r = \frac{\gamma-1}{\gamma+1}$.
- (d) Deduce $f(\mathbf{x}_k) = r^{2k} f(\mathbf{x}_0)$ and compare it with the Kantorovich bound.
- (e) How many iterations reduce f by a factor 10^6 when $\gamma = 10$, and when $\gamma = 100$?

What this practises: a derivation, a proof by induction, and how conditioning sets the rate. The closed form in (c) is a classical example (Boyd & Vandenberghe [4], Section 9.3.2).

(a) Let $\varphi(\alpha) = f(\mathbf{x} - \alpha \mathbf{g}) = \frac{1}{2}(\mathbf{x} - \alpha \mathbf{g})^T \mathbf{A}(\mathbf{x} - \alpha \mathbf{g})$, where $\mathbf{g} = \mathbf{A}\mathbf{x}$.

$$\varphi'(\alpha) = -\mathbf{g}^T \mathbf{A}(\mathbf{x} - \alpha \mathbf{g}) = -\mathbf{g}^T \mathbf{g} + \alpha \mathbf{g}^T \mathbf{A} \mathbf{g} = 0 \implies \alpha_k = \frac{\mathbf{g}_k^T \mathbf{g}_k}{\mathbf{g}_k^T \mathbf{A} \mathbf{g}_k}$$

Since $\varphi''(\alpha) = \mathbf{g}^T \mathbf{A} \mathbf{g} > 0$, this is a minimum.

(b) From $\mathbf{g}_{k+1} = \mathbf{A}\mathbf{x}_{k+1} = \mathbf{g}_k - \alpha_k \mathbf{A} \mathbf{g}_k$,

$$\mathbf{g}_k^T \mathbf{g}_{k+1} = \mathbf{g}_k^T \mathbf{g}_k - \alpha_k \mathbf{g}_k^T \mathbf{A} \mathbf{g}_k = 0 \quad \blacksquare$$

This is true for any smooth f

Exact line search means $\varphi'(\alpha_k) = -\nabla f(\mathbf{x}_{k+1})^T \mathbf{g}_k = 0$. So each step is at a **right angle** to the one before it, and that is where the zig-zag comes from.

(c) Induction. The base case $k = 0$ is $\mathbf{x}_0 = (\gamma, 1)^\top$. Now assume $\mathbf{x}_k = (\gamma r^k, (-r)^k)$. Here $\mathbf{A} = \text{diag}(1, \gamma)$, so

$$\mathbf{g}_k = (\gamma r^k, \gamma(-r)^k), \quad \mathbf{g}_k^\top \mathbf{g}_k = 2\gamma^2 r^{2k}, \quad \mathbf{g}_k^\top \mathbf{A} \mathbf{g}_k = \gamma^2 r^{2k} (1 + \gamma).$$

The step turns out to be *the same every time*: $\alpha_k = \frac{2}{1 + \gamma}$. Then

$$x_{k+1} = \gamma r^k \left(1 - \frac{2}{1+\gamma}\right) = \gamma r^k \cdot \frac{\gamma-1}{\gamma+1} = \gamma r^{k+1},$$

$$y_{k+1} = (-r)^k \left(1 - \frac{2\gamma}{1+\gamma}\right) = (-r)^k \cdot \frac{1-\gamma}{1+\gamma} = (-r)^{k+1}. \quad \blacksquare$$

The y -coordinate changes sign at every step: that is the zig-zag. For $\gamma = 10$, $\mathbf{x}_1 = (8.18, -0.818)$.

$$(d) f(\mathbf{x}_k) = \frac{1}{2}(\gamma^2 r^{2k} + \gamma r^{2k}) = r^{2k} \cdot \frac{1}{2}(\gamma^2 + \gamma) = \boxed{r^{2k} f(\mathbf{x}_0)}.$$

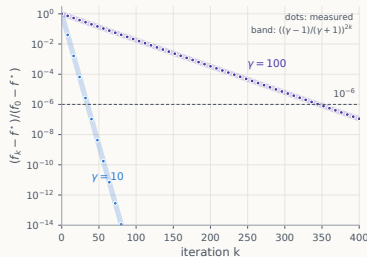
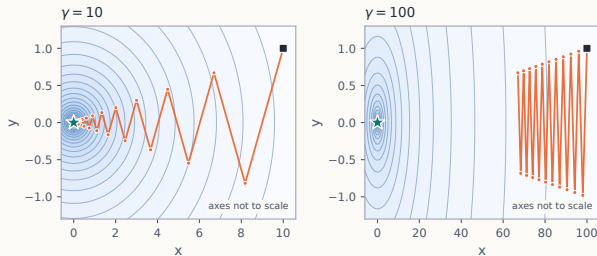
The Kantorovich bound for steepest descent with exact line search says

$f_{k+1} - f^* \leq \left(\frac{\kappa-1}{\kappa+1}\right)^2 (f_k - f^*)$, and here $\kappa = \gamma$. This example **hits the bound exactly**, so the bound cannot be improved.

$$(e) r^{2k} \leq 10^{-6} \iff k \geq \frac{3 \ln 10}{\ln \frac{\gamma+1}{\gamma-1}}:$$

$$\gamma = 10 : k \geq \frac{6.9078}{0.20067} = 34.4 \Rightarrow \boxed{35}, \quad \gamma = 100 : k \geq \frac{6.9078}{0.020001} = 345.4 \Rightarrow \boxed{346}.$$

For large γ , $\ln \frac{\gamma+1}{\gamma-1} \approx \frac{2}{\gamma}$, so $k \approx 3.45 \gamma$. **The work grows in proportion to the condition number.**



Takeaways

- Even the *best* step length along $-\mathbf{g}$ is slow: the **direction** is the problem, which is why we need momentum (M5) and preconditioning (A1, A2).
- Start on an eigenvector, say $(1, 0)$, and one step lands on the minimum. The worst case depends on $\mathbf{x}_0$.

Problem M5

Mathematical ★★

The heavy-ball (Polyak) iteration is $\mathbf{x}_{k+1} = \mathbf{x}_k - \alpha \nabla f(\mathbf{x}_k) + \beta(\mathbf{x}_k - \mathbf{x}_{k-1})$. Model each eigen-direction by $f(x) = \frac{1}{2}\lambda x^2$ with $\lambda \in [\mu, L]$.

- (a) Show that $x_{k+1} = (1 + \beta - \alpha\lambda)x_k - \beta x_{k-1}$, and derive its characteristic polynomial.
- (b) Show that when the roots are complex, $|r_1| = |r_2| = \sqrt{\beta}$, *whatever λ is*.
- (c) For $\mu = 1$, $L = 100$, take $\alpha = \frac{4}{(\sqrt{L} + \sqrt{\mu})^2}$ and $\beta = \left(\frac{\sqrt{L} - \sqrt{\mu}}{\sqrt{L} + \sqrt{\mu}}\right)^2$. Compute them, and show that every $\lambda \in [1, 100]$ gives complex or repeated roots.
- (d) How many iterations do GD (best $\alpha = \frac{2}{\mu+L}$) and heavy ball need to cut the error by 10^6 ?
- (e) What goes on at exactly $\lambda = \mu$?

(a) With $\nabla f = \lambda x$: $x_{k+1} = x_k - \alpha \lambda x_k + \beta x_k - \beta x_{k-1} = (1 + \beta - \alpha \lambda) x_k - \beta x_{k-1}$.

Trying $x_k = r^k$ gives the **characteristic polynomial**

$$r^2 - (1 + \beta - \alpha \lambda) r + \beta = 0, \quad \text{with companion matrix } \mathbf{T} = \begin{pmatrix} 1 + \beta - \alpha \lambda & -\beta \\ 1 & 0 \end{pmatrix}.$$

The iteration converges iff the spectral radius $\rho(\mathbf{T}) = \max |r_i|$ is below 1.

(b) Vieta gives $r_1 r_2 = \beta$. If the roots are complex conjugates, $r_2 = \bar{r}_1$, so

$$|r_1|^2 = r_1 \bar{r}_1 = r_1 r_2 = \beta \implies |r_{1,2}| = \sqrt{\beta}.$$

The roots are complex exactly when the discriminant is negative:

$$(1 + \beta - \alpha \lambda)^2 < 4\beta \iff (1 - \sqrt{\beta})^2 < \alpha \lambda < (1 + \sqrt{\beta})^2.$$

(c) Here $\kappa = L/\mu = 100$ and $\sqrt{\kappa} = 10$, so

$$\alpha = \frac{4}{(10+1)^2} = \frac{4}{121} \approx 0.03306, \quad \sqrt{\beta} = \frac{9}{11}, \quad \beta = \frac{81}{121} \approx 0.6694.$$

The edges of the complex region are

$$(1 - \sqrt{\beta})^2 = \left(\frac{2}{11}\right)^2 = \frac{4}{121} = \alpha\mu, \quad (1 + \sqrt{\beta})^2 = \left(\frac{20}{11}\right)^2 = \frac{400}{121} = \alpha L.$$

So as λ runs over $[1, 100]$, $\alpha\lambda$ covers *exactly* $[\frac{4}{121}, \frac{400}{121}]$: complex roots inside, a double root at each end. Therefore

$$\rho(\mathbf{T}) = \sqrt{\beta} = \frac{9}{11} \approx 0.818 \quad \text{for every } \lambda \in [1, 100].$$

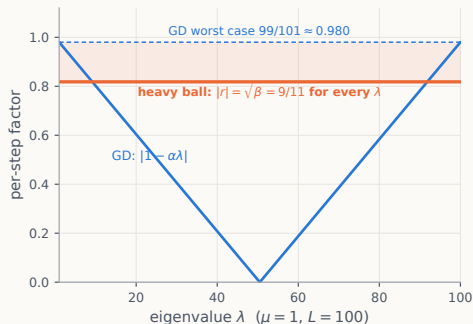
That is the whole point of this tuning: it puts the entire spectrum on the circle $|r| = \sqrt{\beta}$.

(d) Contraction factors, and iterations from $k \geq \ln 10^6 / \ln(1/\rho)$:

$$\rho_{\text{GD}} = \frac{\kappa-1}{\kappa+1} = \frac{99}{101}, \quad k_{\text{GD}} \geq \frac{13.8155}{0.0200} \Rightarrow \boxed{691}$$

$$\rho_{\text{HB}} = \frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1} = \frac{9}{11}, \quad k_{\text{HB}} \geq \frac{13.8155}{0.2007} \Rightarrow \boxed{69}$$

That is about $10 = \sqrt{\kappa}$ times faster. In general the cost drops from $O(\kappa \log \frac{1}{\varepsilon})$ to $O(\sqrt{\kappa} \log \frac{1}{\varepsilon})$.



(e) Repeated roots at the ends. At $\lambda = \mu$ the discriminant is 0 and there is a double root $r = \frac{1}{2}(1 + \beta - \alpha\mu) = \sqrt{\beta} = \frac{9}{11}$, so

$$x_k = (c_1 + c_2 k) r^k \quad (\text{an extra factor of } k \text{ appears}).$$

- So the error is *not* bounded by $1 \cdot r^k$. From $x_0 = x_{-1} = 1$ with $\lambda = \mu$, heavy ball needs 83 iterations for $|x_k| < 10^{-6}$, not 69 (checked numerically); GD needs 691.
- Rates like ρ_{HB} are *asymptotic*: $\|\mathbf{e}_k\| \leq C(k) \rho^k$, where $C(k)$ grows at most polynomially.

Beyond quadratics, be careful

For a general strongly convex f , heavy ball with these parameters is not guaranteed to converge. Lessard, Recht and Packard give a counterexample [14]; Nesterov's method has a proof.

Problem A1 · Linear regression by GD, and why feature scaling matters

Problem A1

Application: Machine Learning ★★

A model $\hat{y} = wx + b$ is trained on the data $(1, 2)$, $(2, 3)$, $(3, 5)$ by minimising the mean-squared-error loss

$$\mathcal{L}(w, b) = \frac{1}{2n} \sum_{i=1}^n (wx_i + b - y_i)^2, \quad n = 3.$$

- (a) Derive $\nabla \mathcal{L}$ and take two GD steps from $(0, 0)$ with $\alpha = 0.1$.
- (b) Find the Hessian, its eigenvalues, κ , and the largest stable α .
- (c) Replace x by the centred feature $\tilde{x} = x - \bar{x}$. Recompute the Hessian and κ , and compare the iterations needed for 10^{-6} accuracy at the best fixed step.
- (d) Find the optimal (w, b) and explain what you see.

The point: training *is* gradient descent, and preprocessing is preconditioning.

(a) Gradient. With residuals $r_i = wx_i + b - y_i$,

$$\frac{\partial \mathcal{L}}{\partial w} = \frac{1}{n} \sum_i r_i x_i, \quad \frac{\partial \mathcal{L}}{\partial b} = \frac{1}{n} \sum_i r_i.$$

Step 1 from $(0, 0)$: $r = (-2, -3, -5)$ and $\mathcal{L} = 6.3333$.

$$\nabla \mathcal{L} = \left(\frac{-2-6-15}{3}, \frac{-10}{3} \right) = (-7.6667, -3.3333) \Rightarrow (w_1, b_1) = (0.7667, 0.3333)$$

and the loss drops to $\mathcal{L}(w_1, b_1) = 1.2826$. **Step 2:** the predictions are $(1.1000, 1.8667, 2.6333)$, so $r = (-0.9000, -1.1333, -2.3667)$.

$$\nabla \mathcal{L} = \left(\frac{-0.9-2.2667-7.1}{3}, \frac{-4.4}{3} \right) = (-3.4222, -1.4667)$$

$$(w_2, b_2) = (1.1089, 0.4800), \quad \mathcal{L} = 0.2807.$$

(b) Hessian. $\mathcal{L}$ is quadratic, so the Hessian is constant: $\mathbf{H} = \frac{1}{n} \sum_i \begin{pmatrix} x_i^2 & x_i \\ x_i & 1 \end{pmatrix} = \begin{pmatrix} 14/3 & 2 \\ 2 & 1 \end{pmatrix}$.

$$\lambda^2 - \frac{17}{3}\lambda + \frac{2}{3} = 0 \Rightarrow \lambda = \frac{17 \pm \sqrt{265}}{6}, \text{ so}$$

$$\lambda_{\max} = 5.5465, \quad \lambda_{\min} = 0.1202, \quad \kappa \approx 46.1, \quad \alpha < \frac{2}{\lambda_{\max}} \approx 0.3606$$

The best fixed step is $\alpha^* = \frac{2}{\lambda_{\min} + \lambda_{\max}} = \frac{6}{17} \approx 0.353$. Its rate is $\frac{\kappa-1}{\kappa+1} = 0.9576$, which means **319 iterations** to reach 10^{-6} . For three data points!

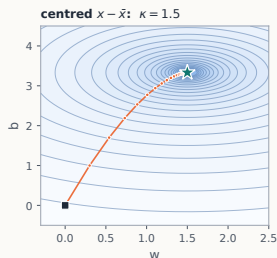
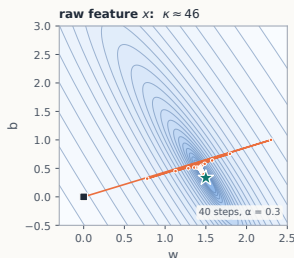
Why is such a tiny problem so badly conditioned?

Every x_i is positive ($\bar{x} = 2$), so the columns $[x, 1]$ are strongly correlated. Nudging w or nudging b changes the fit in almost the same way, and that makes a long, narrow valley.

(c) **Centring.** $\tilde{x} = x - 2 = (-1, 0, 1)$, so $\bar{\tilde{x}} = 0$ and $\overline{\tilde{x}^2} = \frac{2}{3}$:

$$\tilde{H} = \begin{pmatrix} 2/3 & 0 \\ 0 & 1 \end{pmatrix} \Rightarrow \kappa = 1.5, \quad \alpha^* = \frac{2}{2/3 + 1} = 1.2, \quad \rho = \frac{0.5}{2.5} = 0.2.$$

Iterations for 10^{-6} : $\lceil \ln 10^6 / \ln 5 \rceil = 9$ instead of 319. One subtraction made it 35 times faster.



(d) The optimum. Setting $\nabla \mathcal{L} = 0$ gives the normal equations (using $\sum x_i = 6$, $\sum x_i^2 = 14$, $\sum y_i = 10$, $\sum x_i y_i = 23$):

$$14w + 6b = 23, \quad 6w + 3b = 10 \implies w = 1.5, \quad b = \frac{1}{3}, \quad \mathcal{L}^* = \frac{1}{36} \approx 0.0278.$$

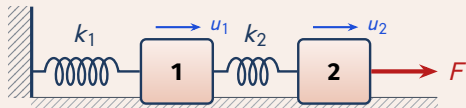
The centred model gives $\hat{y} = 1.5(x - 2) + \frac{10}{3}$, which is *the same line*. Centring changes the shape of the optimisation problem, not its answer.

The bigger picture

- Centring and standardising features is a change of variables that acts as a *preconditioner*: GD sees nearly round contours instead of a narrow valley.
- Here a closed form exists. With millions of parameters there is none, GD or SGD is the only practical solver, and conditioning decides how long training takes.

Problem A2 · Where does a spring chain settle? Energy minimisation

Problem A2



Application: Structural Mechanics ★★

Two masses are joined by springs and a load F pulls on mass 2. The chain comes to rest where the potential energy is smallest:

$$\Pi(\mathbf{u}) = \frac{1}{2}k_1u_1^2 + \frac{1}{2}k_2(u_2 - u_1)^2 - Fu_2.$$

- (a) Write $\Pi = \frac{1}{2}\mathbf{u}^T\mathbf{K}\mathbf{u} - \mathbf{f}^T\mathbf{u}$ and show that $\nabla\Pi = 0$ is force balance.
- (b) With $k_1 = 3$, $k_2 = 1$ kN/mm and $F = 1$ kN, find $\mathbf{u}^*$, $\text{eig}(\mathbf{K})$, κ , the stable range of α , the best α and its rate.
- (c) Take two GD iterations from $\mathbf{u}_0 = 0$ with $\alpha = 0.4$.
- (d) Make the support very stiff, $k_1 = 1000$. What changes, and why is this a *stiff* problem?

(a) Expand $\frac{1}{2}k_2(u_2^2 - 2u_1u_2 + u_1^2)$ to read off

$$\mathbf{K} = \begin{pmatrix} k_1 + k_2 & -k_2 \\ -k_2 & k_2 \end{pmatrix}, \quad \mathbf{f} = \begin{pmatrix} 0 \\ F \end{pmatrix}, \quad \nabla \Pi = \mathbf{K}\mathbf{u} - \mathbf{f}.$$

Setting each component to zero:

$$(k_1 + k_2)u_1 - k_2u_2 = 0 \iff k_1u_1 = k_2(u_2 - u_1) \quad (\text{balance at mass 1})$$

$$k_2(u_2 - u_1) = F \iff \text{spring 2 carries the load} \quad (\text{balance at mass 2})$$

What GD is doing physically

$-\nabla \Pi = \mathbf{f} - \mathbf{K}\mathbf{u}$ is the **unbalanced force** on each mass, and GD nudges each mass along it: relaxation in pseudo-time. Since $\mathbf{K} \succ 0$, there is exactly one resting position.

(b) $\mathbf{K} = \begin{pmatrix} 4 & -1 \\ -1 & 1 \end{pmatrix}$ and $\det \mathbf{K} = 3$:

$$\mathbf{u}^* = \mathbf{K}^{-1} \mathbf{f} = \frac{1}{3} \begin{pmatrix} 1 & 1 \\ 1 & 4 \end{pmatrix} \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \left(\frac{1}{3}, \frac{4}{3} \right) \text{ mm}$$

Sanity check with springs in series: $u_1 = F/k_1 = \frac{1}{3}$ and $u_2 = u_1 + F/k_2 = \frac{4}{3} \checkmark$.

$$\lambda^2 - 5\lambda + 3 = 0 \Rightarrow \lambda = \frac{5 \pm \sqrt{13}}{2} = 0.6972, 4.3028, \text{ so } \kappa = 6.17.$$

$$0 < \alpha < \frac{2}{4.3028} = 0.4648, \quad \alpha^* = \frac{2}{\lambda_1 + \lambda_2} = 0.4, \quad \rho = \frac{\sqrt{13}}{5} = 0.7211.$$

Getting to 10^{-6} accuracy takes $\lceil \ln 10^6 / \ln(1/0.7211) \rceil = 43$ iterations.

(c) Two iterations with $\alpha = 0.4$ and $\mathbf{g}_k = \mathbf{K}\mathbf{u}_k - \mathbf{f}$:

k	$\mathbf{u}_k$ (mm)	$\mathbf{g}_k$	$\Pi(\mathbf{u}_k)$
0	(0, 0)	(0, -1)	0
1	(0, 0.4)	(-0.4, -0.6)	-0.3200
2	(0.16, 0.64)	(0, -0.52)	-0.4864
★	(0.3333, 1.3333)	(0, 0)	-0.6667

Insight: news travels one spring per iteration

After step 1 only mass 2 has moved; mass 1 “hears” about the load at step 2. Since $\mathbf{u}_k \in \text{span}\{\mathbf{f}, \mathbf{K}\mathbf{f}, \dots, \mathbf{K}^{k-1}\mathbf{f}\}$ and $\mathbf{K}$ is tridiagonal, a chain of N springs needs at least N iterations before every mass has moved.

(d) A stiff support, $k_1 = 1000$: $\mathbf{K} = \begin{pmatrix} 1001 & -1 \\ -1 & 1 \end{pmatrix}$.

$$\lambda \approx 0.999, 1001.001 \Rightarrow \kappa \approx 1002, \quad \alpha_{\max} \approx 0.0020, \quad \rho \approx 0.998.$$

Iterations for 10^{-6} : about 6922 instead of 43, for the easy answer $\mathbf{u}^* = (0.001, 1.001)$ mm.

- The stiff spring forces a tiny α for stability, and then the soft mode crawls. GD is explicit Euler on $\dot{\mathbf{u}} = -(\mathbf{K}\mathbf{u} - \mathbf{f})$, whose eigenvalues span three orders of magnitude. That is a textbook stiff ODE.
- The fix is Jacobi scaling: $\mathbf{D}^{-1/2}\mathbf{K}\mathbf{D}^{-1/2}$ with $\mathbf{D} = \text{diag}(\mathbf{K})$ has eigenvalues $1 \pm \frac{1}{\sqrt{1001}}$, so $\kappa \approx 1.065$.

Engineering lesson

Stiff members next to soft ones make the landscape ill-conditioned: rescale before using a first-order solver.

Problem A3 · Maximising a two-product profit by gradient ascent

Problem A3

Application: Economics ★★

A firm sells two substitute products in quantities x and y (hundreds of units). Its profit, in thousand Tk, is

$$P(x, y) = 100x + 80y - 2x^2 - 2y^2 - 2xy.$$

Each period the firm adjusts production by *gradient ascent*, $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha \nabla P(\mathbf{x}_k)$.

- (a) Find the optimum analytically and show that it is a global maximum.
- (b) Find the stable range of α and the best fixed α .
- (c) Run three iterations from $(0, 0)$ with the best α , and tabulate P and the error.
- (d) Explain what $\alpha = 0.4$ does, in business terms.

The point: ascent is just descent on $-P$, and a hand calculation matches the theory exactly.

$$(a) \nabla P = \begin{pmatrix} 100 - 4x - 2y \\ 80 - 2x - 4y \end{pmatrix} = 0 \iff \begin{cases} 4x + 2y = 100 \\ 2x + 4y = 80 \end{cases} \iff (x^*, y^*) = (20, 10)$$

$$\nabla^2 P = \begin{pmatrix} -4 & -2 \\ -2 & -4 \end{pmatrix}, \text{ eigenvalues } -2, -6 \Rightarrow \text{negative definite} \Rightarrow \text{global maximum}$$

$$P^* = 2000 + 800 - 800 - 200 - 400 = 1400 \text{ thousand Tk.}$$

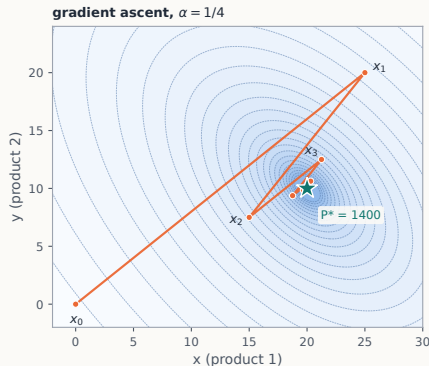
$$(b) \text{ Ascent on } P \text{ is GD on } -P, \text{ with } \mathbf{A} = \begin{pmatrix} 4 & 2 \\ 2 & 4 \end{pmatrix}: \lambda = 2 \text{ along } (1, -1) \text{ and } \lambda = 6 \text{ along } (1, 1).$$

$$0 < \alpha < \frac{2}{6} = \frac{1}{3}, \quad \alpha^* = \frac{2}{2+6} = \frac{1}{4}, \quad \rho = \frac{6-2}{6+2} = \frac{1}{2}.$$

(c) $\alpha = \frac{1}{4}$, starting from $(0, 0)$:

k	$\mathbf{x}_k$	$\nabla P(\mathbf{x}_k)$	P	$\ \mathbf{e}_k\ $
0	(0, 0)	(100, 80)	0	22.361
1	(25, 20)	(-40, -50)	1050	11.180
2	(15, 7.5)	(25, 20)	1312.5	5.590
3	(21.25, 12.5)	(-10, -12.5)	1378.125	2.795

- The error **halves exactly** every step, because both factors, $1 - \frac{1}{4} \cdot 2 = \frac{1}{2}$ and $1 - \frac{1}{4} \cdot 6 = -\frac{1}{2}$, have size $\frac{1}{2}$.
- The profit gap shrinks by $\rho^2 = \frac{1}{4}$ per step:
 $1400 \rightarrow 350 \rightarrow 87.5 \rightarrow 21.875$.



The overshoot pattern. The $(1, 1)$ part of the error changes sign every step (its factor is $-\frac{1}{2}$). So total production first *overshoots* ($25 + 20 = 45 > 30$), then *undershoots*, then overshoots again, by half as much each time.

(d) $\alpha = 0.4 > \frac{1}{3}$. Now the factor along $(1, 1)$ is $1 - 0.4 \cdot 6 = -1.4$, and $\mathbf{x}_6 \approx (-92.9, -102.9)$. The swings grow instead of dying out. In business terms: a management that *over-reacts* to the marginal-profit signal pushes total output further from the optimum every period, until it is planning negative (meaningless) quantities.

Lesson

How fast you adjust has to respect the curvature of the profit function: $\alpha < 2/\lambda_{\max}$. Because the goods are substitutes (the $-2xy$ term), $\lambda_{\max}$ rises from 4 to 6 and the safe step gets smaller.

Problem A4 · Estimating a cooling constant by nonlinear least squares

Problem A4

Application: Parameter Estimation ★★★

By Newton's law of cooling, the normalised temperature $\theta = \frac{T - T_a}{T_0 - T_a}$ follows $\theta(t) = e^{-kt}$. We measure $\theta(1) = 0.60$ and $\theta(2) = 0.36$ (t in minutes) and estimate k by minimising

$$J(k) = \frac{1}{2} \sum_{i=1}^2 (e^{-kt_i} - \theta_i)^2.$$

- (a) Derive $J'(k)$ and $J''(k)$, and find the exact minimiser k^* .
- (b) Run two GD iterations from $k_0 = 0$ with $\alpha = 0.5$. Does GD converge?
- (c) Compare the curvature J'' at $k = 0$ and at k^* . Where is J convex?
- (d) Now start from a poor guess, $k_0 = 6$. What does GD do, and how would you fix it?

The point: a nonconvex fit, curvature that varies, and the plateau failure mode.

(a) With residuals $r_i(k) = e^{-kt_i} - \theta_i$,

$$J'(k) = - \sum_i r_i t_i e^{-kt_i}, \quad J''(k) = \underbrace{\sum_i t_i^2 e^{-2kt_i}}_{\text{Gauss-Newton part } \geq 0} + \underbrace{\sum_i r_i t_i^2 e^{-kt_i}}_{\text{residual part (either sign)}}.$$

The data fit the model perfectly: $e^{-k} = 0.6$ and $e^{-2k} = 0.36 = 0.6^2$. So $J(k^*) = 0$ at

$$k^* = \ln \frac{5}{3} \approx 0.5108 \text{ min}^{-1}.$$

Why this is not a linear problem

J depends on k through e^{-kt} , so it is *not* quadratic. Its curvature changes with k , which means there is no longer one step size that is safe everywhere.

(b) Two GD iterations with $\alpha = 0.5$:

- $k_0 = 0$: $r = (0.4, 0.64)$, $J'(0) = -(0.4 \cdot 1 + 0.64 \cdot 2) = -1.68$, so $k_1 = 0 + 0.5(1.68) = 0.84$.
- $k_1 = 0.84$: $e^{-0.84} = 0.43171$, $e^{-1.68} = 0.18637$, $r = (-0.16829, -0.17363)$, and

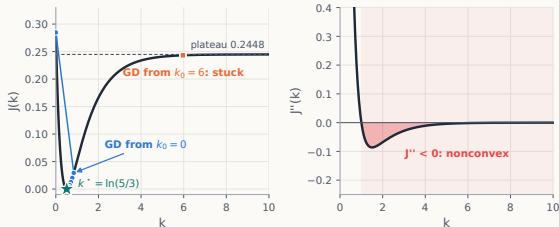
$$J'(0.84) = -[(-0.16829)(1)(0.43171) + (-0.17363)(2)(0.18637)] = 0.13737$$

$$\text{so } k_2 = 0.84 - 0.5(0.13737) = 0.7713.$$

$$J : 0.2848 \rightarrow 0.02923 \rightarrow 0.02015 \quad (\text{going down})$$

The steep gradient at $k = 0$ makes the first step **overshoot** k^* ; after that the iterates come back from above. GD reaches $k^* = 0.510826$ to within 10^{-6} in 26 iterations.

(c) **Curvature.** $J''(0) = (1 + 0.4) + (4 + 4 \cdot 0.64) = 7.96$,
 $J''(k^*) = 0.36 + 4(0.1296) = 0.8784$ (the residuals vanish there).



- The local stability limit $2/J''$ is 0.25 at $k = 0$ but 2.28 at k^* , a ninefold change. Near k^* the rate is $|1 - 0.5(0.8784)| = 0.56$ per step.
- For $k \gtrsim 1.00$ (shaded) $J'' < 0$, so J is **nonconvex** there and no global step-size result applies.

(d) A bad start, $k_0 = 6$. Here $e^{-6} = 0.00248$ and $e^{-12} \approx 6 \times 10^{-6}$, so the model says the object is “already cold” at both measurement times:

$$J(6) = 0.2433 \approx \frac{1}{2} \sum \theta_i^2 = 0.2448 \text{ (plateau)}, \quad J'(6) \approx 1.5 \times 10^{-3}.$$

Each step moves k by only about 7×10^{-4} . After **1000 iterations** we are at $k \approx 4.65$. GD has **stalled on a plateau**: the gradient vanishes because the model has saturated.

How to get unstuck

- **Start somewhere sensible.** Linearising, $\ln \theta = -k t$ gives $k \approx -\ln 0.6 = 0.51$.
- **Reparametrise**, e.g. $k = e^s$, or fit $\ln \theta$ instead, which reshapes the landscape.
- **Adapt the step** with a line search or curvature scaling, so it does not stay tiny on flat ground.

Problem A5

Application: Signal Processing ★★

A temperature sensor reports $\xi_k = \theta + \varepsilon_k$, where the ε_k are i.i.d. with mean 0 and variance $\sigma^2 = 0.25$ ($^{\circ}\text{C}$)². We estimate θ by minimising $F(x) = \frac{1}{2}\mathbb{E}[(x - \xi)^2]$, using *one new reading per step*:

$$x_{k+1} = x_k - \alpha_k (x_k - \xi_{k+1}) \quad (\text{stochastic gradient descent}).$$

- (a) Show that $\nabla F(x) = x - \theta$ and that the stochastic gradient $x - \xi$ is unbiased.
- (b) Show that $\alpha_k = \frac{1}{k+1}$ with $x_1 = \xi_1$ gives the running sample mean, so $\text{Var}(x_k) = \sigma^2/k$.
- (c) For a constant α , show that $\text{Var}(x_k) \rightarrow \alpha\sigma^2/(2 - \alpha)$.
- (d) With readings 20.4, 19.7, 20.2, 20.6, 19.9, run both schedules (constant $\alpha = 0.5$).
- (e) Connect the results to the Robbins–Monro step-size conditions.

(a) Expand, using $\mathbb{E}\xi = \theta$ and $\text{Var } \xi = \sigma^2$:

$$F(x) = \frac{1}{2}\mathbb{E}[(x - \theta - \varepsilon)^2] = \frac{1}{2}(x - \theta)^2 + \frac{1}{2}\sigma^2 \implies \nabla F(x) = x - \theta, \quad x^* = \theta.$$

So F is a quadratic with $\mu = L = 1$. The stochastic gradient $G(x, \xi) = x - \xi$ has $\mathbb{E}[G] = x - \theta = \nabla F(x)$: it is **unbiased**, and it needs only one reading instead of the whole expectation.

(b) **Induction.** Suppose $x_k = \frac{1}{k} \sum_{i \leq k} \xi_i$. Then

$$x_{k+1} = x_k + \frac{\xi_{k+1} - x_k}{k+1} = \frac{k x_k + \xi_{k+1}}{k+1} = \frac{1}{k+1} \sum_{i \leq k+1} \xi_i. \quad \blacksquare$$

The readings are independent, so $\text{Var}(x_k) = \sigma^2/k \rightarrow 0$. With this schedule SGD is the sample mean.

(c) **Constant step.** Subtract θ from both sides:

$$x_{k+1} - \theta = (1 - \alpha)(x_k - \theta) + \alpha \varepsilon_{k+1}.$$

- **Mean:** $\mathbb{E}[x_k] - \theta = (1 - \alpha)^k(x_0 - \theta) \rightarrow 0$ geometrically, so the bias disappears.
- **Variance:** ε_{k+1} is independent of x_k , so $V_{k+1} = (1 - \alpha)^2 V_k + \alpha^2 \sigma^2$. Its fixed point is

$$V_\infty = \frac{\alpha^2 \sigma^2}{1 - (1 - \alpha)^2} = \frac{\alpha^2 \sigma^2}{\alpha(2 - \alpha)} = \boxed{\frac{\alpha \sigma^2}{2 - \alpha}} \quad (0 < \alpha < 2).$$

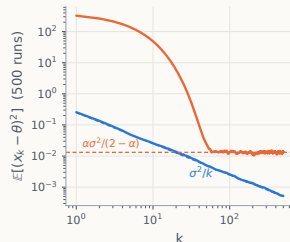
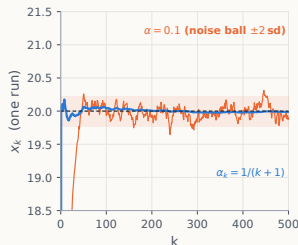
The noise ball

Constant-step SGD gets close fast, then jitters in a *neighbourhood* of radius $O(\sqrt{\alpha})$. Monte-Carlo, $\alpha = 0.1$: $V_\infty = 0.01320$ against 0.01316 predicted.

(d) Both schedules start from $x_1 = \xi_1 = 20.4$:

k	ξ_k	$\alpha_k = \frac{1}{k+1}$	$\alpha = 0.5$
1	20.4	20.4000	20.4000
2	19.7	20.0500	20.0500
3	20.2	20.1000	20.1250
4	20.6	20.2250	20.3625
5	19.9	20.1600	20.1313

Standard deviations: the sample mean after 5 readings has $\sqrt{0.25/5} = 0.224$ and it keeps shrinking; $\alpha = 0.5$ is stuck at $\sqrt{0.5 \cdot 0.25/1.5} = 0.289$.



A constant step weights old readings by $\alpha(1 - \alpha)^j$, so it slowly forgets them. That is useful if θ drifts, and wasteful if it does not.

(e) The Robbins–Monro conditions:

$$\sum_k \alpha_k = \infty \quad \text{and} \quad \sum_k \alpha_k^2 < \infty.$$

- $\sum \alpha_k = \infty$: the steps add up to enough “travel” to forget the start, since $\prod(1 - \alpha_k) \rightarrow 0$.
- $\sum \alpha_k^2 < \infty$: the total injected noise variance $\sum \alpha_k^2 \sigma^2$ stays finite and gets averaged away.

Schedule ($k \geq 1$)	$\sum \alpha_k$	$\sum \alpha_k^2$	What happens
$\alpha_k = \frac{1}{k+1}$	∞	$\frac{\pi^2}{6} - 1$	converges to θ ✓
$\alpha_k = \alpha$	∞	∞	noise ball, $V_\infty = \frac{\alpha \sigma^2}{2 - \alpha}$
$\alpha_k = \frac{1}{(k+1)^2}$	$< \infty$	$< \infty$	stops moving too early

For $\frac{1}{(k+1)^2}$, $\prod_{k \geq 1} (1 - \frac{1}{(k+1)^2}) = \frac{1}{2}$: only half of any initial error ever gets removed.

Beyond this example.

- **Mini-batches:** averaging B readings per step divides the noise variance by B :

$$V_{\infty} = \frac{\alpha \sigma^2}{B(2 - \alpha)}.$$

This is the step-size against batch-size trade-off behind large-scale machine learning.

- **Any strongly convex F :** with a fixed step, the expected optimality gap of SGD falls linearly to a floor proportional to α . With diminishing steps $\alpha_k = O(1/k)$ it decays like $O(1/k)$ [3].

Takeaway

A decaying schedule gives up speed to gain accuracy. A constant step gives up accuracy to gain speed and the ability to follow changes. Practical schedules (constant first, then decaying) get some of both.



References



-
- [1] Larry Armijo. **Minimization of functions having Lipschitz continuous first partial derivatives.** *Pacific Journal of Mathematics*, 16(1):1–3, 1966.
 - [2] Jonathan Barzilai and Jonathan M. Borwein. **Two-point step size gradient methods.** *IMA Journal of Numerical Analysis*, 8(1):141–148, 1988.
 - [3] Léon Bottou, Frank E. Curtis, and Jorge Nocedal. **Optimization methods for large-scale machine learning.** *SIAM Review*, 60(2):223–311, 2018.
 - [4] Stephen Boyd and Lieven Vandenberghe. **Convex Optimization.** Cambridge University Press, 2004.
 - [5] Sébastien Bubeck. **Convex optimization: Algorithms and complexity.** *Foundations and Trends in Machine Learning*, 8(3–4):231–357, 2015.

-
- [6] Augustin-Louis Cauchy. **Méthode générale pour la résolution des systèmes d'équations simultanées.** *Comptes Rendus Hebdomadaires des Séances de l'Académie des Sciences*, 25:536–538, 1847.
 - [7] Steven C. Chapra and Raymond P. Canale. **Numerical Methods for Engineers.** McGraw-Hill Education, 8th edition, 2020.
 - [8] Jeremy M. Cohen, Simran Kaur, Yuanzhi Li, J. Zico Kolter, and Ameet Talwalkar. **Gradient descent on neural networks typically occurs at the edge of stability.** In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2103.00065.
 - [9] Gabriel Goh. **Why momentum really works.** Distill, 2017.
 - [10] Andreas Griewank and Andrea Walther. **Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation.** SIAM, 2nd edition, 2008.

-
- [11] Hamed Karimi, Julie Nutini, and Mark Schmidt. **Linear convergence of gradient and proximal-gradient methods under the Polyak–Łojasiewicz condition.** In *Machine Learning and Knowledge Discovery in Databases (ECML PKDD 2016)*, volume 9851 of *Lecture Notes in Computer Science*, pages 795–811. Springer, 2016.
- [12] Diederik P. Kingma and Jimmy Ba. **Adam: A method for stochastic optimization.** In *3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.
- [13] Jason D. Lee, Max Simchowitz, Michael I. Jordan, and Benjamin Recht. **Gradient descent only converges to minimizers.** In *Proceedings of the 29th Conference on Learning Theory (COLT)*, volume 49 of *Proceedings of Machine Learning Research*, pages 1246–1257, 2016.

-
- [14] Laurent Lessard, Benjamin Recht, and Andrew Packard. **Analysis and design of optimization algorithms via integral quadratic constraints.** *SIAM Journal on Optimization*, 26(1):57–95, 2016.
- [15] Yurii Nesterov. **Lectures on Convex Optimization.** Springer, 2nd edition, 2018.
- [16] Yurii E. Nesterov. **A method for solving the convex programming problem with convergence rate $O(1/k^2)$.** *Soviet Mathematics Doklady*, 27(2):372–376, 1983.
- [17] Jorge Nocedal and Stephen J. Wright. **Numerical Optimization.** Springer Series in Operations Research and Financial Engineering. Springer, 2nd edition, 2006.
- [18] Boris T. Polyak. **Some methods of speeding up the convergence of iteration methods.** *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.

-
- [19] Herbert Robbins and Sutton Monro. **A stochastic approximation method.** *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [20] Weijie Su, Stephen Boyd, and Emmanuel J. Candès. **A differential equation for modeling Nesterov’s accelerated gradient method: Theory and insights.** *Journal of Machine Learning Research*, 17(153):1–43, 2016.